Daftar isi

C++17 | Klik materi untuk membuka halaman. Range [l,r) mencakup l sampai r-1; indeks mulai 0.

Materi	Hal.
Prefix, pencarian & greedy	2
Fenwick, DSU & sparse table	3
Segment tree & lazy affine	4
Persistent, query 2D & small/large	5
Trie XOR, basis & bit tricks	6
Shortest path & pemodelan state	7
SCC, bridge, DAG & spanning tree	8
LCA, tree DP & dominator DAG	9
Bipartite matching, flow & trie string	10
DP dasar, urutan & state kecil	12
Bitmask, profile DP, SOS & game	13

Materi	Hal.
Teori bilangan, modular & CRT	14
Kombinatorika, ekspektasi & matriks	15
Geometri integer & convex hull	16
String: KMP, Z, palindrome & hash	17
Suffix array, LCP & ranking substring	18
NTT & konvolusi siklik	19
Formal power series & generating function	20
INC 2021: interval constraints & teleport DP	21
INC 2025: observasi & query khusus	22
INC 2022-2024: recurrence & konstruksi	23
Tambahan CP & merge interval AND	24

Persiapan kode: #include <bits/stdc++.h> dan using namespace std; gunakan alias ll=long long, i128=__int128_t, ull=unsigned long long. MOD=998244353; INF=4e18 (integer). WGraph=vector<vector<pair<int,ll>>>. Header pendamping sudah memuat definisi dan dependensi: #include "inc_templates.hpp" lalu using namespace inc;. ckmin(a,b)/ckmax(a,b) memperbarui a bila b lebih kecil/besar. PBDS memerlukan header ext/pb_ds.

Prefix, pencarian & greedy

Prefix sum 1D / 2D

Fungsi / syarat: Build O(nm), query kotak [r1,r2) x [c1,c2): O(1).

Cara kerja: Simpan jumlah dari awal; kurangi dua prefix untuk range, atau gunakan inclusion-exclusion empat sudut untuk kotak.

Pakai: auto p=prefix(a); p[r]-p[l] menjumlahkan a[l..r-1]. Prefix2D q(grid); q.get(r1,c1,r2,c2) menjumlahkan kotak dengan batas kanan/bawah eksklusif.

```
struct Prefix2D {
  vector<vector<ll>> p;
  Prefix2D(const vector<vector<ll>> &a) {
    int n = a.size(), m = n ? a[0].size() : 0;
    p.assign(n + 1, vector<ll>(m + 1));
    for (int i = 0; i < n; i++)
      for (int j = 0; j < m; j++)
        p[i + 1][j + 1] = a[i][j] + p[i][j + 1] +
                          p[i + 1][j] - p[i][j];
  }
  ll get(int r1, int c1, int r2, int c2) {
    return p[r2][c2] - p[r1][c2] - p[r2][c1] +
           p[r1][c1];
  }
};
vector<ll> prefix(const vector<ll> &a) {
  vector<ll> p(a.size() + 1);
  for (int i = 0; i < (int)a.size(); i++)
    p[i + 1] = p[i] + a[i];
  return p; // sum[l,r) = p[r]-p[l]
}
```

Binary search jawaban

Fungsi / syarat: firstTrue: predicate false lalu true; O(log(R-L)).

Cara kerja: Buang setengah rentang berdasarkan ok(mid); jawaban harus berubah dari false ke true, tidak bolak-balik.

Pakai: ll x=firstTrue(0,hi,can); can(x) menguji apakah x cukup. Hasil hi berarti tidak ada solusi di [0,hi); buat hi lebih besar dari semua kandidat.

```
template <class Predicate>
ll firstTrue(ll left, ll right, Predicate can) {
  // Cari pada [left, right); right adalah sentinel.
  while (left < right) {
    ll middle = left + (ll)(((i128)right - left) / 2);
    if (can(middle)) {
      right = middle;
    } else {
      left = middle + 1;
    }
  }
  return left;
}
```

Binary search jawaban (lanjutan)

Subarray jumlah <= K

Fungsi / syarat: HANYA a[i]>=0, K>=0. O(n).

Cara kerja: Perbesar ujung kanan; geser kiri sampai sum<=K. Sebanyak r-l+1 subarray valid berakhir di r.

Pakai: countAtMost(a,K) mengembalikan banyak subarray tidak kosong. Contoh {1,2,1}, K=3 menghasilkan 5; hanya untuk elemen nonnegatif.

```
ll countAtMost(const vector<ll> &a, ll limit) {
  if (limit < 0) return 0;
  ll windowSum = 0;
  ll answer = 0;
  int left = 0;
  for (int right = 0; right < (int)a.size();
       right++) {
    windowSum += a[right];
    while (windowSum > limit) {
      windowSum -= a[left];
      left++;
    }
    answer += right - left + 1;
  }
  return answer;
}
```

Subarray jumlah tepat K

Fungsi / syarat: Menangani negatif. O(n log n).

Cara kerja: Untuk prefix saat ini s, setiap prefix sebelumnya bernilai s-K membentuk satu subarray berjumlah K.

Pakai: countSum({1,-1,1},1) menghasilkan 3. Map menyimpan frekuensi prefix, termasuk prefix kosong 0; query dilakukan sebelum menambah prefix sekarang.

```
ll countSum(const vector<ll> &a, ll target) {
  map<ll, ll> frequency;
  frequency[0] = 1; // Prefix kosong.
  ll prefixSum = 0;
  ll answer = 0;
  for (ll value : a) {
    prefixSum += value;
    answer += frequency[prefixSum - target];
    frequency[prefixSum]++;
  }
  return answer;
}
```

Sliding minimum + monotonic deque

Fungsi / syarat: Jendela panjang k, 1<=k<=n. O(n).

Cara kerja: Deque menyimpan kandidat minimum yang nilainya menaik. Stack nextSmaller membuang indeks saat menemukan nilai lebih kecil.

Pakai: windowMin({3,1,2},2) -> {1,1}. nextSmaller(a)[i] adalah indeks pertama di kanan dengan nilai <a[i], atau n bila tidak ada.

```
vector<ll> windowMin(const vector<ll> &a, int k) {
  deque<int> q;
  vector<ll> ans;
  for (int i = 0; i < (int)a.size(); i++) {
    while (!q.empty() && q.front() <= i - k)
      q.pop_front();
    while (!q.empty() && a[q.back()] >= a[i])
      q.pop_back();
    q.push_back(i);
    if (i + 1 >= k) ans.push_back(a[q.front()]);
  }
  return ans;
}
vector<int> nextSmaller(const vector<ll> &a) {
  int n = a.size();
  vector<int> ans(n, n), st;
  for (int i = 0; i < n; i++) {
    while (!st.empty() && a[st.back()] > a[i]) {
      ans[st.back()] = i;
      st.pop_back();
    }
    st.push_back(i);
  }
  return ans;
}
```

Greedy assignment: kapasitas & reward

Fungsi / syarat: Item (ukuran, reward), satu item per slot. O(n log n).

Cara kerja: Urutkan slot kecil dahulu; masukkan semua item yang muat ke max-heap, lalu ambil reward positif terbesar.

Pakai: assignReward({{2,5},{3,8}},{2,3}) -> 13. Item adalah {ukuran,reward}; cap berisi kapasitas setiap slot, dan slot boleh tidak diisi.

```
ll assignReward(vector<pair<ll, ll>> item,
                vector<ll> cap) {
  sort(item.begin(), item.end());
  sort(cap.begin(), cap.end());
  priority_queue<ll> q;
  ll ans = 0;
  int j = 0;
  for (ll c : cap) {
    while (j < (int)item.size() && item[j].first <= c)
```

Greedy assignment: kapasitas & reward (lanjutan)

```
      q.push(item[j++].second);
    if (!q.empty() && q.top() > 0) {
      ans += q.top();
      q.pop();
    }
  }
  return ans;
}
```

Pola greedy & sweep

Interval tidak bertumpuk: urutkan ujung kanan lalu ambil yang mulai >= ujung sebelumnya. Cover [L,R]: dari interval dengan kiri <= posisi, pilih kanan terjauh. Sweep event: definisikan urutan start/end saat koordinat sama. Interval inclusive dan half-open membutuhkan tie rule berbeda.

Selisih, kontribusi, marginal cost

Jumlah subarray memuat i = (i+1)*(n-i). Jumlah semua panjang subarray ukuran n = n(n+1)(n+2)/6. Biaya kuadrat a*k^2 memiliki tambahan a*(2k-1); binary search ambang biaya lalu koreksi item yang berlebih (2023-K).

Fenwick, DSU & sparse table

Fenwick: point add / range sum / kth

Fungsi / syarat: O(log n); prefix(r) untuk [0,r). kth perlu frekuensi >=0.

Cara kerja: BIT menyimpan jumlah blok ukuran lowbit. Prefix mengunjungi blok-blok penyusun range; kth turun menurut jumlah kumulatif.

Pakai: Fenwick f(n); f.add(i,delta); ll s=f.sum(l,r). f.kth(k) memberi indeks elemen ke-k dalam multiset frekuensi, k mulai 1. inversions(a) menghitung pasangan i<j dengan a[i]>a[j].

```
struct Fenwick {
  int n;
  vector<ll> bit;
  Fenwick(int n) : n(n), bit(n + 1) {}
  void add(int p, ll v) {
    for (++p; p <= n; p += p & -p)
      bit[p] += v;
  }
  ll pref(int r) const {
    ll s = 0;
    for (; r > 0; r -= r & -r)
      s += bit[r];
    return s;
  }
  ll sum(int l, int r) const {
```

Fenwick: point add / range sum / kth (lanjutan)

```
    return pref(r) - pref(l);
  }
  int kth(ll k) const { // k=1..total
    int p = 0, step = 1;
    while (step <= n / 2)
      step *= 2;
    for (; step; step >>= 1) {
      int q = p + step;
      if (q <= n && bit[q] < k) {
        p = q;
        k -= bit[q];
      }
    }
    return p; // 0-based; n jika total<k
  }
};
ll inversions(vector<ll> a) {
  vector<ll> b = a;
  sort(b.begin(), b.end());
  b.erase(unique(b.begin(), b.end()), b.end());
  Fenwick f(b.size());
  ll ans = 0;
  for (int i = 0; i < (int)a.size(); i++) {
    int p = lower_bound(b.begin(), b.end(), a[i]) -
            b.begin();
    ans += i - f.pref(p + 1);
    f.add(p, 1);
  }
  return ans;
}
```

Fenwick range add / range sum

Fungsi / syarat: Dua BIT; add [l,r), query prefix [0,r). O(log n).

Cara kerja: Dua BIT menyimpan difference array dan koreksi berbobot indeks; pref(r)=r*A(r)-B(r).

Pakai: RangeBIT f(5); f.add(1,4,2); f.sum(0,5) -> 6. Array awal semua nol; add(l,r,v) menambah v pada setiap elemen [l,r). Dependensi: Fenwick.

```
struct RangeBIT {
  Fenwick
      difference; // Menyimpan delta pada batas range.
  Fenwick weighted; // Menyimpan index * delta.
  // Extra satu posisi untuk batas kanan r == n.
  RangeBIT(int n)
      : difference(n + 1), weighted(n + 1) {}
  // Tambah value ke setiap elemen dalam [left,
  // right).
  void add(int left, int right, ll value) {
    difference.add(left, value);
    difference.add(right, -value);
```

Fenwick range add / range sum (lanjutan)

```
    weighted.add(left, value * left);
    weighted.add(right, -value * right);
  }
  // Jumlah elemen pada [0, right).
  ll pref(int right) {
    ll deltaSum = difference.pref(right);
    ll weightedSum = weighted.pref(right);
    return right * deltaSum - weightedSum;
  }
  ll sum(int left, int right) {
    return pref(right) - pref(left);
  }
};
```

DSU / Union-Find

Fungsi / syarat: Amortized O(alpha(n)); komponen, Kruskal, merge interval.

Cara kerja: Setiap komponen memiliki root. Path compression mempercepat find; union by size menempelkan komponen kecil ke besar.

Pakai: DSU d(n); bool merged=d.join(u,v); merged=false bila sudah tersambung. d.find(u)==d.find(v) menguji konektivitas; d.sz[d.find(u)] memberi ukuran komponen.

```
struct DSU {
  vector<int> parent;
  vector<int> sz; // Ukuran komponen, valid pada root.
  DSU(int n) : parent(n), sz(n, 1) {
    iota(parent.begin(), parent.end(), 0);
  }
  int find(int node) {
    if (parent[node] == node) return node;
    parent[node] = find(parent[node]);
    return parent[node];
  }
  bool join(int a, int b) {
    int rootA = find(a);
    int rootB = find(b);
    if (rootA == rootB) return false;
    if (sz[rootA] < sz[rootB]) swap(rootA, rootB);
    parent[rootB] = rootA;
    sz[rootA] += sz[rootB];
    return true;
  }
};
```

Sparse table RMQ minimum

Fungsi / syarat: Statis, n>=1; build O(n log n), query nonempty O(1).

Cara kerja: Precompute minimum blok panjang 2^k; dua blok yang menutup query boleh overlap karena min idempotent.

Pakai: SparseMin st(a); st.get(1,4) adalah min(a[1],a[2],a[3]). Array tidak boleh berubah setelah build; wajib l<r.

```
struct SparseMin {
  vector<vector<ll>> t;
  SparseMin(const vector<ll> &a) {
    int n = a.size();
    assert(n);
    int K = 32 - __builtin_clz(n);
    t.assign(K, vector<ll>(n));
    t[0] = a;
    for (int k = 1; k < K; k++)
      for (int i = 0; i + (1 << k) <= n; i++)
        t[k][i] = min(t[k - 1][i],
                      t[k - 1][i + (1 << (k - 1))]);
  }
  ll get(int l, int r) const {
    assert(l < r);
    int k = 31 - __builtin_clz(r - l);
    return min(t[k][l], t[k][r - (1 << k)]);
  }
};
```

Order-statistics tree (GCC PBDS)

Fungsi / syarat: O(log n); pair(value,id) untuk nilai duplikat.

Cara kerja: Balanced BST diberi ukuran subtree sehingga dapat menghitung rank dan mencari elemen berdasarkan urutan.

Pakai: OrderedSet<pair<ll,int>> s; s.insert({5,0}); s.insert({5,1}); s.order_of_key({5,-1}) -> 0. s.find_by_order(1)->first -> 5. ID unik mencegah duplikat hilang.

```
template <class T>
using OrderedSet = __gnu_pbds::tree<
    T,
    __gnu_pbds::null_type,
    less<T>,
    __gnu_pbds::rb_tree_tag,
    __gnu_pbds::tree_order_statistics_node_update>;
```

Pilih struktur

Prefix: statis sum. Fenwick: update titik, agregat invertible. Sparse: statis min/max/gcd. Segment tree: monoid umum / update lebih kaya. Persistent tree: simpan banyak versi. Sqrt decomposition: pisahkan parameter kecil dan besar.

Segment tree & lazy affine

Segment tree generik

Fungsi / syarat: Monoid: op asosiatif + identitas e. O(log n) per operasi.

Cara kerja: Daun menyimpan nilai; parent menggabungkan anak dengan op. Query mengumpulkan blok kiri dan kanan sambil menjaga urutannya.

Pakai: SegTree<ll,SumOp> st(n,0,{}); st.setval(i,x); ll z=st.get(l,r). Nilai awal 0; untuk min ganti Op dan identitas menjadi INF.

```
template <class T, class Op> struct SegTree {
  int size;
  T identity;
  Op combine;
  vector<T> tree;
  SegTree(int n, T identity, Op combine)
      : identity(identity), combine(combine) {
    size = 1;
    while (size < n)
      size *= 2;
    tree.assign(2 * size, identity);
  }
  void setval(int index, T value) {
    int node = index + size;
    tree[node] = value;
    for (node /= 2; node > 0; node /= 2) {
      tree[node] =
          combine(tree[2 * node], tree[2 * node + 1]);
    }
  }
  T get(int left, int right) const {
    T resultLeft = identity;
    T resultRight = identity;
    left += size;
    right += size;
    while (left < right) {
      if (left & 1)
        resultLeft =
            combine(resultLeft, tree[left++]);
      if (right & 1)
        resultRight =
            combine(tree[--right], resultRight);
      left /= 2;
      right /= 2;
    }
    return combine(resultLeft, resultRight);
  }
};
struct SumOp {
  ll operator()(ll a, ll b) const {
    return a + b;
```

Segment tree generik (lanjutan)

```
  }
};
```

Komposisi fungsi domain 0..15

Fungsi / syarat: INC 2025-E. Urutan merge: kanan(kiri(x)), bukan sebaliknya.

Cara kerja: Setiap modul disimpan sebagai tabel hasil untuk 16 input. Merge a,b membentuk fungsi b(a(x)).

Pakai: Func16 id; iota(id.begin(),id.end(),0); SegTree<Func16,Compose16> st(n,id,{}); st.setval(i,module16(type,k)); st.get(l,r)[x] memberi keluaran input x setelah modul [l,r).

```
using Func16 = array<int, 16>;
struct Compose16 {
  Func16 operator()(Func16 a, Func16 b) const {
    Func16 c;
    for (int x = 0; x < 16; x++)
      c[x] = b[a[x]];
    return c;
  }
};
Func16 module16(char type, int k) {
  Func16 f;
  for (int x = 0; x < 16; x++)
    f[x] = (type == '+') ? (x + k) % 16 : (x ^ k);
  return f;
}
```

Desain node & lazy

Node menyimpan informasi yang bisa digabung: sum; (min,count); (sum,prefix,suffix,best) untuk max-subarray; matriks; tabel fungsi. Jika update tidak dapat diterapkan ke node tanpa melihat anak, tambah informasi node atau ganti pendekatan.

Range maksimum subarray

merge L,R: sum=L.sum+R.sum; pre=max(L.pre,L.sum+R.pre); suf=max(R.suf,R.sum+L.suf); best=max(L.best,R.best,L.suf+R.pre). Tentukan apakah subarray kosong boleh; ini mengubah identitas.

Lazy: x <- a*x+b (mod MOD)

Fungsi / syarat: Range add (1,b), multiply (a,0), assign (0,b). O(log n).

Cara kerja: Simpan sum node serta transform tertunda. Transform baru a*x+b digabung ke tag lama; push hanya saat turun ke anak.

Pakai: LazyAffine st(n); st.upd(i,i+1,0,a[i]) mengisi nilai awal. st.upd(l,r,1,5) tambah 5; (2,0) kali 2; (0,7) assign 7. st.sum(l,r) mengembalikan jumlah mod MOD; normalisasi parameter negatif.

Lazy: x <- a*x+b (mod MOD) (lanjutan)

```
struct LazyAffine {
  int n;
  vector<ll> s, m, b;
  LazyAffine(int n)
      : n(n), s(4 * n), m(4 * n, 1), b(4 * n) {
    assert(n > 0);
  }
  void apply(int p, int len, ll a, ll c) {
    s[p] = (a * s[p] + c * len) % MOD;
    m[p] = a * m[p] % MOD;
    b[p] = (a * b[p] + c) % MOD;
  }
  void push(int p, int l, int r) {
    int mid = (l + r) / 2;
    apply(p * 2, mid - l, m[p], b[p]);
    apply(p * 2 + 1, r - mid, m[p], b[p]);
    m[p] = 1;
    b[p] = 0;
  }
  void
  upd(int L, int R, ll a, ll c, int p, int l, int r) {
    if (R <= l || r <= L) return;
    if (L <= l && r <= R) {
      apply(p, r - l, a, c);
      return;
    }
    push(p, l, r);
    int mid = (l + r) / 2;
    upd(L, R, a, c, p * 2, l, mid);
    upd(L, R, a, c, p * 2 + 1, mid, r);
    s[p] = (s[p * 2] + s[p * 2 + 1]) % MOD;
  }
  ll ask(int L, int R, int p, int l, int r) {
    if (R <= l || r <= L) return 0;
    if (L <= l && r <= R) return s[p];
    push(p, l, r);
    int mid = (l + r) / 2;
    return (ask(L, R, p * 2, l, mid) +
            ask(L, R, p * 2 + 1, mid, r)) %
           MOD;
  }
  void upd(int l, int r, ll a, ll c) {
    a = (a % MOD + MOD) % MOD;
    c = (c % MOD + MOD) % MOD;
    upd(l, r, a, c, 1, 0, n);
  }
  ll sum(int l, int r) {
    return ask(l, r, 1, 0, n);
  }
};
```

Komposisi tag

Tag lama (m,b), lalu baru (a,c): menjadi (a*m,a*b+c). Assign bukan add; gunakan (0,x), sehingga nilai 0 juga valid. Untuk sum integer biasa, hapus modulo dan pastikan semua hasil tidak overflow.

Pencarian dalam tree

Jika node menyimpan sum frekuensi nonnegatif, kth turun kiri bila sumLeft>=k; jika tidak, k-=sumLeft dan turun kanan. Jika node menyimpan max, cari pertama >=x dengan melewati node yang max<x. Keduanya O(log n).

Persistent, query 2D & small/large

Persistent segment tree: frekuensi

Fungsi / syarat: Versi prefix; O(log V) update/kth, O(n log V) node.

Cara kerja: Update menyalin hanya jalur root-ke-daun; versi lama tetap utuh. Selisih dua root memberi frekuensi subarray.

Pakai: Persistent pst; vector<int> root(n+1); root[i+1]=pst.add(root[i],0,V,rank[i]); pst.kth(root[l],root[r],0,V,k) memberi rank ke-k, 1<=k<=r-l; petakan kembali ke nilai asli.

```
struct Persistent {
  struct Node {
    int l = 0, r = 0, c = 0;
  };
  vector<Node> t = {{}};
  int add(int old, int l, int r, int x) {
    int p = t.size();
    t.push_back(t[old]);
    t[p].c++;
    if (r - l > 1) {
      int m = (l + r) / 2;
      if (x < m)
        t[p].l = add(t[old].l, l, m, x);
      else
        t[p].r = add(t[old].r, m, r, x);
    }
    return p;
  }
  int count(int p, int l, int r, int L, int R) {
    if (!p || R <= l || r <= L) return 0;
    if (L <= l && r <= R) return t[p].c;
    int m = (l + r) / 2;
    return count(t[p].l, l, m, L, R) +
           count(t[p].r, m, r, L, R);
  }
  int kth(int a, int b, int l, int r, int k) {
    // versi b minus versi a; 1<=k<=selisih
    if (r - l == 1) return l;
    int c = t[t[b].l].c - t[t[a].l].c;
```

Persistent segment tree: frekuensi (lanjutan)

```
    int m = (l + r) / 2;
    if (k <= c) return kth(t[a].l, t[b].l, l, m, k);
    return kth(t[a].r, t[b].r, m, r, k - c);
  }
};
```

Small/large parameter untuk query modulo

Fungsi / syarat: INC 2022-I; add O(B), query O(1) atau O(n/k).

Cara kerja: Simpan total tiap residue untuk modulus kecil. Modulus besar dijawab dengan melompat indeks rem,rem+k,... sehingga hanya sedikit posisi.

Pakai: ResidueSums rs(n); rs.add(i,delta); rs.get(rem,k) menjumlahkan semua posisi i%k=rem. Array awal nol; 0<=rem<k, k>=1.

```
struct ResidueSums {
  int n, B;
  vector<ll> a;
  vector<vector<ll>> s;
  ResidueSums(int n) : n(n), B(sqrt(n) + 1), a(n) {
    s.resize(B);
    for (int k = 1; k < B; k++)
      s[k].assign(k, 0);
  }
  void add(int i, ll v) {
    a[i] += v;
    for (int k = 1; k < B; k++)
      s[k][i % k] += v;
  }
  ll get(int rem, int k) {
    assert(k > 0 && 0 <= rem && rem < k);
    if (k < B) return s[k][rem];
    ll ans = 0;
    for (int i = rem; i < n; i += k)
      ans += a[i];
    return ans;
  }
};
```

Merge-sort tree: count dan successor

Fungsi / syarat: Statis, build O(n log n); query O(log^2 n).

Cara kerja: Tiap node segment tree menyimpan daftar terurut. Query range dipecah ke node lalu memakai binary search pada daftarnya.

Pakai: MergeTree st(a); st.lessThan(l,r,x) menghitung nilai <x. st.nextGE(l,r,x) memberi nilai terkecil >=x, atau INF. Nilai dan indeks array tetap setelah build.

```
struct MergeTree {
  int n;
```

Merge-sort tree: count dan successor (lanjutan)

```
  vector<vector<ll>> t;
  MergeTree(vector<ll> a) {
    n = 1;
    while (n < (int)a.size())
      n *= 2;
    t.resize(n * 2);
    for (int i = 0; i < (int)a.size(); i++)
      t[n + i] = {a[i]};
    for (int i = n - 1; i; i--) {
      auto &a = t[2 * i], &b = t[2 * i + 1];
      merge(a.begin(),
            a.end(),
            b.begin(),
            b.end(),
            back_inserter(t[i]));
    }
  }
  int lessThan(int l, int r, ll x) {
    int ans = 0;
    auto take = [&](int p) {
      ans +=
          lower_bound(t[p].begin(), t[p].end(), x) -
          t[p].begin();
    };
    for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
      if (l & 1) take(l++);
      if (r & 1) take(--r);
    }
    return ans;
  }
  ll nextGE(int l, int r, ll x) {
    ll ans = INF;
    auto take = [&](int p) {
      auto it =
          lower_bound(t[p].begin(), t[p].end(), x);
      if (it != t[p].end()) ckmin(ans, *it);
    };
    for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
      if (l & 1) take(l++);
      if (r & 1) take(--r);
    }
    return ans;
  }
};
```

Geometri statis menjadi query range

Urutkan titik berdasarkan x; simpan y di MergeTree. Kotak [x1,x2] x [y1,y2] menjadi range indeks x dan count y. Untuk 2021-D: cek titik ketiga di dalam kotak dua titik (count>=3); cek 4 strip memakai successor/predecessor; 4 kuadran memakai sweep.

Mo: kapan dipakai

Query offline [l,r), update satu elemen ke state murah. Urutkan (l/B, r naik/turun bergantian), B sekitar n/sqrt(q). Geser L/R sambil add/remove. O((n+q)*sqrt(n)) pada skala seimbang. Tidak cocok jika predicate perlu urutan asli yang mahal dipelihara.

Interval dinamis

Simpan map<left,right> blok aktif. Toggle: cari containing interval dengan prev(upper_bound(x)); pecah saat off, gabung tetangga saat on. Sebelum ubah blok kurangi kontribusinya; setelah ubah tambahkan. 2022-J memakai f(len)=len(len+1)(len+2)/6.

Trie XOR, basis & bit tricks

Binary trie dengan count

Fungsi / syarat: B=30: nilai [0,2^30). add +/-1 O(B); maxXor O(B).

Cara kerja: Trie bercabang menurut bit. maxXor memilih bit berlawanan; mexXor melewati subtree penuh untuk menemukan bilangan hilang pertama.

Pakai: XTrie t; t.add(5,1); t.maxXor(2) -> 7 (nilai XOR, bukan elemennya). t.add(5,-1) menghapus. maxXor perlu trie tidak kosong; mexXor(x) perlu SET tanpa duplikat.

```
struct XTrie {
  static const int B = 30;
  struct Node {
    int ch[2] = {0, 0};
    int cnt = 0;
  };
  vector<Node> t = vector<Node>(2); // root=1
  void add(int x, int delta) {
    int u = 1;
    t[u].cnt += delta;
    for (int b = B - 1; b >= 0; b--) {
      int k = x >> b & 1;
      if (!t[u].ch[k]) {
        t[u].ch[k] = t.size();
        t.push_back({});
      }
      u = t[u].ch[k];
      t[u].cnt += delta;
    }
  }
  int maxXor(int x) {
    assert(t[1].cnt);
    int u = 1, ans = 0;
    for (int b = B - 1; b >= 0; b--) {
      int k = x >> b & 1;
      if (t[t[u].ch[k ^ 1]].cnt) {
        ans |= 1 << b;
        k ^= 1;
      }
      u = t[u].ch[k];
    }
    return ans;
  }
  int mexXor(int x) { // SET: tidak ada duplikat
    if (t[1].cnt == (1 << B)) return 1 << B;
    int u = 1, ans = 0;
    for (int b = B - 1; b >= 0; b--) {
      int k = x >> b & 1, v = t[u].ch[k];
      if (t[v].cnt == (1 << b)) {
        ans |= 1 << b;
        k ^= 1;
      }
      u = t[u].ch[k];
    }
    return ans;
  }
};
```

Min MEX sesudah XOR i, 0<=i<=K

Fungsi / syarat: 2025-G: dekomposisi [0,K] ke blok dyadic; O(B^2).

Cara kerja: Pecah pilihan XOR [0,K] menjadi blok pangkat dua; pada tiap blok, bit rendah bebas sehingga minimum MEX bisa diperoleh dari trie.

Pakai: Masukkan tiap nilai berbeda ke XTrie t, lalu minMexBound(t,K). Hasil adalah min mex({a[i]^x}) untuk 0<=x<=K; semua nilai dan K harus <2^30.

```
int minMexBound(XTrie &t, int K) {
  if ((ll)t.t[1].cnt < K + 1LL) return 0;
  int ans = INT_MAX, base = 0;
  ll len = K + 1LL;
  for (int b = 30; b >= 0; b--)
    if (len >> b & 1) {
      int z = t.mexXor(base);
      ans = min(ans, (z >> b) << b);
      base += (1 << b);
    }
  return ans;
}
```

XOR linear basis dengan timestamp

Fungsi / syarat: Representasi subset XOR; insert/query O(B), 0<=x<2^30.

Cara kerja: Eliminasi bit tinggi seperti Gaussian elimination di GF(2). Setiap pivot mempertahankan posisi terbaru agar query suffix dapat dilakukan.

Pakai: PosBasis b; b.add(a[R-1],R) saat menyapu R=1..n. b.latestLeft(x)>=L berarti x dapat dibentuk oleh subset [L,R], indeks 1-based; b.maxXor(0,L) mencari XOR maksimum pada rentang itu.

```
struct PosBasis {
  int a[30] = {}, p[30] = {}; // posisi 1-based
  void add(int x, int id) {
    for (int b = 29; b >= 0; b--)
      if (x >> b & 1) {
        if (!a[b]) {
          a[b] = x;
          p[b] = id;
          return;
        }
        if (p[b] < id) {
          swap(a[b], x);
          swap(p[b], id);
        }
        x ^= a[b];
      }
  }
  int latestLeft(int x) const {
    int l = INT_MAX;
    for (int b = 29; b >= 0; b--)
      if (x >> b & 1) {
        if (!a[b]) return 0;
        x ^= a[b];
        l = min(l, p[b]);
      }
    return l;
  }
  int maxXor(int x = 0, int L = 1) const {
    for (int b = 29; b >= 0; b--)
      if (p[b] >= L) x = max(x, x ^ a[b]);
    return x;
  }
};
```

INC 2025-H: shortest feasible subrange

Target=(1<<M)-1. Sweep r, insert mask[r] pada posisi r+1, ambil left[r]=latestLeft(target). Jika 0 berarti belum feasible. left[r] tidak menurun. len[r]=r+2-left[r], atau INF. Query [L,R] 1-based: cari r pertama dengan left[r]>=L; jawab RMQ min len pada [r,R), atau -1. Build O(nM+n log n), query O(log n).

Enumerasi bit / submask

Fungsi / syarat: Pola; semua submask semua mask berbiaya O(3^B).

Cara kerja: Rumus (s-1)&mask mengunjungi semua subset bit; x&=x-1 menghapus bit aktif paling rendah satu per satu.

Pakai: Taruh loop di dalam solve() setelah mendefinisikan mask. Loop pertama termasuk subset kosong; ctz hanya dipanggil saat x!=0. Gunakan unsigned untuk manipulasi lowbit.

```
for (int s = mask;; s = (s - 1) & mask) {
  // gunakan s (termasuk 0)
  if (s == 0) break;
}
for (unsigned x = mask; x; x &= x - 1) {
  int b = __builtin_ctz(x);
  // bit b aktif
}
// popcount: __builtin_popcountll(x)
// lowbit: x & -x; gunakan tipe unsigned.
// AND range bisa berubah hanya O(B) kali
// untuk subarray yang berakhir di indeks tetap.
```

INC 2022-C: signed binary carry

Simpan digit d[x] dalam {-1,0,1} untuk jumlah bertanda 2^x. Tambah tanda s: d[x]+=s. Jika d[x]==2*s, set 0 lalu carry s ke x+1; ulangi. Pelihara jumlah digit nonzero, sehingga cek total nol O(1). Pakai array cukup besar untuk carry.

Pitfall bit

Gunakan 1LL<<b untuk b>=31. XOR basis memilih subset, trie memilih SATU elemen: kegunaan berbeda. MEX bergantung keberadaan nilai, bukan multiplicity; kelola frekuensi dan ubah count trie hanya pada transisi 0<->1.

Shortest path & pemodelan state

BFS multi-source

Fungsi / syarat: Graf tanpa bobot / semua bobot 1. O(V+E).

Cara kerja: Semua sumber masuk queue dengan jarak 0. BFS memproses lapisan berurutan sehingga penemuan pertama memberikan jarak terpendek.

Pakai: auto d=bfs(g,{s1,s2}); d[v] adalah jarak minimum dari salah satu sumber, atau -1 jika tidak terjangkau. g[u] berisi indeks tetangga; untuk undirected tambahkan kedua arah.

```
vector<int> bfs(const vector<vector<int>> &g,
                const vector<int> &src) {
  vector<int> d(g.size(), -1);
  queue<int> q;
  for (int s : src)
    if (d[s] < 0) {
      d[s] = 0;
      q.push(s);
    }
  while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (int v : g[u])
      if (d[v] < 0) {
        d[v] = d[u] + 1;
        q.push(v);
      }
  }
  return d;
}
```

0-1 BFS

Fungsi / syarat: Hanya bobot 0 atau 1; O(V+E).

Cara kerja: Relaksasi bobot 0 dimasukkan ke depan deque, bobot 1 ke belakang; jarak kecil diproses lebih dahulu.

Pakai: WGraph g(n); g[u].push_back({v,0}); auto d=bfs01(g,s); d[t] memberi jarak atau INF. Tidak boleh ada bobot selain 0 dan 1.

```
vector<ll> bfs01(const WGraph &g, int s) {
  vector<ll> d(g.size(), INF);
  deque<pair<ll, int>> q;
  d[s] = 0;
  q.push_back({0, s});
  while (!q.empty()) {
    auto [du, u] = q.front();
    q.pop_front();
    if (du != d[u]) continue;
    for (auto [v, w] : g[u])
      if (ckmin(d[v], du + w)) {
        if (w == 0)
          q.push_front({d[v], v});
        else
          q.push_back({d[v], v});
      }
  }
  return d;
}
```

Dijkstra

Fungsi / syarat: Bobot >=0; O((V+E) log V). INF harus > semua jarak.

Cara kerja: Min-heap selalu mengambil jarak terkecil. Relaksasi memperbaiki tetangga; entri heap yang sudah usang dilewati.

Pakai: WGraph g(n); g[u].push_back({v,w}); auto d=dijkstra(g,s); d[t]==INF berarti tidak terjangkau. Tambah edge balik jika undirected; semua w>=0.

```
vector<ll> dijkstra(const WGraph &g, int s) {
  vector<ll> d(g.size(), INF);
  d[s] = 0;
  using Q = pair<ll, int>;
  priority_queue<Q, vector<Q>, greater<Q>> q;
  q.push({0, s});
  while (!q.empty()) {
    auto [du, u] = q.top();
    q.pop();
    if (du != d[u]) continue;
    for (auto [v, w] : g[u])
      if (w < INF - du && ckmin(d[v], du + w))
        q.push({d[v], v});
  }
  return d;
}
```

Floyd-Warshall

Fungsi / syarat: All-pairs; O(n^3), memory O(n^2).

Cara kerja: Pada tahap k, izinkan vertex k menjadi perantara: d[i][j]=min(d[i][j],d[i][k]+d[k][j]).

Pakai: Buat Matrix jarak dengan diagonal 0, edge minimum, sisanya INF; panggil floyd(d). Baca d[u][v]. Bobot negatif boleh, tetapi jarak tidak terdefinisi jika lintasan dapat melewati negative cycle.

```
void floyd(vector<vector<ll>> &d) {
  int n = d.size();
  for (int k = 0; k < n; k++)
    for (int i = 0; i < n; i++)
      if (d[i][k] < INF)
        for (int j = 0; j < n; j++)
          if (d[k][j] < INF)
            ckmin(d[i][j], d[i][k] + d[k][j]);
}
```

Bellman-Ford & cycle negatif

Inisialisasi d[s]=0. Relaksasi semua edge (u,v,w) sebanyak n-1 putaran, hanya jika d[u]!=INF. Relaksasi masih berhasil pada putaran n berarti negative cycle reachable. O(VE). SPFA tidak punya jaminan lebih baik; jangan jadikan default.

KHUSUS 2021-I: node (kota,identifier)

Buat node untuk setiap pasangan (kota,p) yang muncul pada edge, plus (1,1),(N,1). Di setiap kota, sort p dan sambungkan pasangan BERSEBELAHAN dengan bobot selisih p. Edge jalan menyambungkan (u,p)<->(v,p) berbobot t. Dijkstra pada O(N+M) state/edge setelah deduplikasi. Tidak perlu clique semua identifier.

KHUSUS 2025-D: tiga shortest path

Fungsi / syarat: g memakai w; h memakai w+ceil(w/2). O((V+E)log V).

Cara kerja: Coba semua titik percabangan v. Gabungkan jarak dari 0 dan 2 pada bobot asli, serta dari 1 pada bobot gabungan.

Pakai: ll ans=divido(g); vertex soal 1,2,3 dipetakan menjadi 0,1,2. g undirected connected, bobot nonnegatif. Hasil adalah biaya minimum sesuai model Divido; dependensi dijkstra.

```
ll divido(const WGraph &g) {
  WGraph h = g;
  for (auto &row : h)
    for (auto &e : row)
      e.second += (e.second + 1) / 2;
  auto a = dijkstra(g, 0), b = dijkstra(h, 1);
  auto c = dijkstra(g, 2);
  ll ans = INF;
  for (int v = 0; v < (int)g.size(); v++)
    if (a[v] < INF && b[v] < INF && c[v] < INF)
      ckmin(ans, a[v] + b[v] + c[v]);
  return ans;
}
```

Graph modeling yang sering membuka soal

State=(node,paritas), (node,sisa kupon), (mask,last), (kota,warna). Super-source untuk banyak sumber. Pisah node in/out untuk kapasitas simpul. Pisah edge menjadi node untuk menghitung edge dominator (2023-I). Biaya waktu menunggu sering menjadi max(waktu tiba,batas buka).

Grid

id=r*m+c. Tetangga 4 arah (1,0),(-1,0),(0,1),(0,-1); 8 arah bila diagonal boleh. Flood fill menghitung komponen. Jika update lokal, cari invariant lokal sebelum memakai dynamic connectivity (2025-I).

SCC, bridge, DAG & spanning tree

Kosaraju iteratif: strongly connected components

Fungsi / syarat: O(V+E). Return (jumlah SCC, label topological order).

Cara kerja: DFS pertama mencatat urutan selesai. DFS pada reverse graph dalam urutan terbalik mengumpulkan vertex yang saling terjangkau.

Pakai: auto [k,comp]=scc(g); comp[u]==comp[v] berarti satu SCC. Bangun DAG baru dengan edge comp[u]->comp[v] hanya bila label berbeda; label berada di [0,k).

```
pair<int, vector<int>>
scc(const vector<vector<int>> &g) {
  int n = g.size();
  vector<vector<int>> rg(n);
  for (int u = 0; u < n; u++)
    for (int v : g[u])
      rg[v].push_back(u);
  vector<int> seen(n), order, at(n), c(n, -1);
  for (int s = 0; s < n; s++)
    if (!seen[s]) {
      vector<int> st = {s};
      seen[s] = 1;
      while (!st.empty()) {
        int u = st.back();
        if (at[u] < (int)g[u].size()) {
          int v = g[u][at[u]++];
          if (!seen[v]) {
            seen[v] = 1;
            st.push_back(v);
          }
        } else {
          order.push_back(u);
          st.pop_back();
        }
      }
    }
  reverse(order.begin(), order.end());
  int k = 0;
  for (int s : order)
    if (c[s] < 0) {
      vector<int> st = {s};
      c[s] = k;
      while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        for (int v : rg[u])
          if (c[v] < 0) {
            c[v] = k;
            st.push_back(v);
          }
```

Kosaraju iteratif: strongly connected components (lanjutan)

```
      }
      k++;
    }
  return {k, c};
}
```

2-SAT dari SCC

Fungsi / syarat: Literal 2*i=false, 2*i+1=true. O(V+E).

Cara kerja: Klausa a OR b menjadi implikasi !a->b dan !b->a. Jika variabel dan negasinya satu SCC, tidak ada assignment.

Pakai: TwoSAT sat(n); sat.either(2*u+1,2*v) berarti xu OR !xv. auto val=sat.solve(); val[i] adalah 0/1; kosong berarti gagal untuk n>0. Negasi literal a adalah a^1.

```
struct TwoSAT {
  int n;
  vector<vector<int>> g;
  TwoSAT(int n) : n(n), g(2 * n) {}
  void either(int a, int b) {
    g[a ^ 1].push_back(b);
    g[b ^ 1].push_back(a);
  }
  vector<int> solve() {
    auto [k, c] = scc(g);
    vector<int> ans(n);
    for (int i = 0; i < n; i++) {
      if (c[2 * i] == c[2 * i + 1]) return {};
      ans[i] = c[2 * i + 1] > c[2 * i];
    }
    return ans;
  }
};
```

Bridge & articulation point

Fungsi / syarat: Undirected, edge ID unik. O(V+E); DFS rekursif.

Cara kerja: low[u] menyimpan waktu DFS terkecil yang terjangkau melalui subtree dan back-edge. Perbandingannya dengan tin mendeteksi pemutus.

Pakai: LowLink llk(n); llk.edge(u,v,id); llk.run(); llk.bridges berisi edge ID, llk.cut[u]!=0 menandai articulation point. Edge ID harus unik; graf boleh terputus.

```
struct LowLink {
  vector<vector<pair<int, int>>> g;
  vector<int> tin, low, cut, bridges;
  int timer = 0;
  LowLink(int n) : g(n), tin(n, -1), low(n), cut(n) {}
  void edge(int u, int v, int id) {
    g[u].push_back({v, id});
```

Bridge & articulation point (lanjutan)

```
    g[v].push_back({u, id});
  }
  void dfs(int u, int pe = -1) {
    tin[u] = low[u] = timer++;
    int children = 0;
    for (auto [v, id] : g[u]) {
      if (id == pe) continue;
      if (tin[v] >= 0)
        low[u] = min(low[u], tin[v]);
      else {
        children++;
        dfs(v, id);
        low[u] = min(low[u], low[v]);
        if (low[v] > tin[u]) bridges.push_back(id);
        if (pe != -1 && low[v] >= tin[u]) cut[u] = 1;
      }
    }
    if (pe == -1 && children > 1) cut[u] = 1;
  }
  void run() {
    for (int u = 0; u < (int)g.size(); u++)
      if (tin[u] < 0) dfs(u);
  }
};
```

Topological sort (Kahn)

Fungsi / syarat: O(V+E); order.size()<n berarti ada directed cycle.

Cara kerja: Queue berisi vertex indegree nol. Hapus satu vertex lalu kurangi indegree semua tujuan; vertex siap berikutnya masuk queue.

Pakai: auto order=topo(g); jika order.size()==g.size(), proses DP sesuai order. Ukuran lebih kecil menandakan cycle; g harus directed.

```
vector<int> topo(const vector<vector<int>> &g) {
  vector<int> d(g.size()), ans;
  queue<int> q;
  for (auto &a : g)
    for (int v : a)
      d[v]++;
  for (int i = 0; i < (int)g.size(); i++)
    if (!d[i]) q.push(i);
  while (!q.empty()) {
    int u = q.front();
    q.pop();
    ans.push_back(u);
    for (int v : g[u])
      if (--d[v] == 0) q.push(v);
  }
  return ans;
}
```

Kruskal / MST

Sort edge (w,u,v) menaik. Jika dsu.join(u,v), pilih edge dan tambah w. Connected graph memerlukan n-1 edge terpilih; jika kurang, hasil minimum spanning forest. O(E log E). Maximum spanning tree: urutkan menurun.

Euler trail & orientasi

Undirected Euler: semua derajat genap (sirkuit) atau tepat dua ganjil (trail); semua vertex ber-edge harus connected. Directed: in=out kecuali start +1 dan end -1. Hierholzer memakai stack dan edge ID, O(E). DFS orientation: tree edge turun, back edge ke ancestor; graf connected tanpa bridge bisa diorientasi strongly connected.

LCA, tree DP & dominator DAG

LCA + jarak tree, tanpa recursive DFS

Fungsi / syarat: Tree connected, root bebas. O(n log n); query O(log n).

Cara kerja: Simpan ancestor 2^k dan depth. Samakan depth lalu angkat kedua vertex dari lompatan terbesar sampai parent mereka sama.

Pakai: LCA lc(g,root); lc.get(u,v) -> ancestor bersama terdalam; lc.distance(u,v) -> jumlah bobot path; lc.jump(v,k) -> ancestor k tingkat. Gunakan k<=lc.dep[v].

```
struct LCA {
  int n, K;
  vector<int> dep, par, order;
  vector<ll> dist;
  vector<vector<int>> up;
  LCA(const WGraph &g, int root = 0) {
    n = g.size();
    K = 1;
    while ((1LL << K) <= n)
      K++;
    dep.assign(n, 0);
    dist.assign(n, 0);
    par.assign(n, -1);
    par[root] = root;
    order = {root};
    for (int i = 0; i < (int)order.size(); i++) {
      int u = order[i];
      for (auto [v, w] : g[u])
        if (par[v] < 0) {
          par[v] = u;
          dep[v] = dep[u] + 1;
          dist[v] = dist[u] + w;
          order.push_back(v);
        }
    }
    up.assign(K, par);
```

LCA + jarak tree, tanpa recursive DFS (lanjutan)

```
    for (int k = 1; k < K; k++)
      for (int v = 0; v < n; v++)
        up[k][v] = up[k - 1][up[k - 1][v]];
  }
  int jump(int v, int k) const {
    if (k > dep[v]) return -1;
    for (int b = 0; b < K; b++)
      if (k >> b & 1) v = up[b][v];
    return v;
  }
  int get(int u, int v) const {
    if (dep[u] < dep[v]) swap(u, v);
    u = jump(u, dep[u] - dep[v]);
    if (u == v) return u;
    for (int b = K - 1; b >= 0; b--)
      if (up[b][u] != up[b][v]) {
        u = up[b][u];
        v = up[b][v];
      }
    return par[u];
  }
  ll distance(int u, int v) const {
    return dist[u] + dist[v] - 2 * dist[get(u, v)];
  }
};
```

Tree DP & reroot

Proses reverse(order) untuk subtree. Independent set maximum: take[u]=1+sum skip[v], skip[u]=sum max(take[v],skip[v]). Counting: take=product skip, skip=product(take+skip). Reroot jumlah jarak: ans[v]=ans[u]+(n-2*size[v])*w(u,v).

Produk kecuali satu anak

Bangun prefix/suffix dari kontribusi semua tetangga; kontribusi selain anak i = pref[i]*suf[i+1]. Hindari pembagian modular jika faktor bisa nol. Dipakai reroot 2023-D dan banyak DP tree.

Immediate dominator pada DAG

Fungsi / syarat: HANYA DAG; idom(v)=LCA semua predecessor reachable.

Cara kerja: Proses DAG topologis. Dominator langsung v adalah LCA semua predecessor yang reachable pada dominator tree yang sudah dibangun.

Pakai: auto idom=dagDominator(g,s); idom[s]=s dan vertex unreachable bernilai -1. Mengikuti idom dari v ke s memberi vertex yang wajib dilewati semua lintasan s->v. Dependensi topo.

```
vector<int> dagDominator(const vector<vector<int>> &g,
                         int s) {
  int n = g.size(), K = 1;
```

Immediate dominator pada DAG (lanjutan)

```
  while ((1LL << K) <= n)
    K++;
  vector<vector<int>> pre(n),
      up(K, vector<int>(n, s));
  for (int u = 0; u < n; u++)
    for (int v : g[u])
      pre[v].push_back(u);
  vector<int> d(n), id(n, -1);
  id[s] = s;
  auto lca = [&](int u, int v) {
    if (d[u] < d[v]) swap(u, v);
    int diff = d[u] - d[v];
    for (int b = 0; b < K; b++)
      if (diff >> b & 1) u = up[b][u];
    if (u == v) return u;
    for (int b = K - 1; b >= 0; b--)
      if (up[b][u] != up[b][v]) {
        u = up[b][u];
        v = up[b][v];
      }
    return up[0][u];
  };
  auto ord = topo(g);
  assert((int)ord.size() == n);
  for (int v : ord)
    if (v != s) {
      int p = -1;
      for (int u : pre[v])
        if (id[u] >= 0) p = (p < 0) ? u : lca(p, u);
      if (p < 0) continue;
      id[v] = p;
      d[v] = d[p] + 1;
      up[0][v] = p;
      for (int b = 1; b < K; b++)
        up[b][v] = up[b - 1][up[b - 1][v]];
    }
  return id;
}
```

DP (ways,sum): gabung pilihan independen

Fungsi / syarat: 2023-D. sum = total ukuran / skor semua konfigurasi.

Cara kerja: Penjumlahan menggabungkan alternatif. Perkalian pasangan independen: ways=A.ways*B.ways, sum=A.sum*B.ways+A.ways*B.sum.

Pakai: WaysSum pilih={1,1}, kosong={1,0}; auto opsi=pilih+kosong; auto dua=opsi*opsi; dua.ways=4 dan dua.sum=4. Semua dihitung modulo MOD.

```
struct WaysSum {
  ll ways = 0, sum = 0;
```

DP (ways,sum): gabung pilihan independen (lanjutan)

```
  WaysSum operator+(WaysSum b) const {
    return {(ways + b.ways) % MOD,
            (sum + b.sum) % MOD};
  }
  WaysSum operator*(WaysSum b) const {
    return {ways * b.ways % MOD,
            (sum * b.ways + ways * b.sum) % MOD};
  }
};
```

Subtree, path & diameter

DFS Euler entry tin dan exit tout memetakan subtree ke [tin,tout). Path update dapat dipecah dengan HLD. Pada tree unweighted/nonnegative, dua traversal terjauh menghasilkan diameter; trik ini tidak berlaku untuk graf umum.

Bipartite matching, flow & trie string

Hopcroft-Karp

Fungsi / syarat: Bipartite: kiri [0,L), kanan [0,R). O(E sqrt(V)).

Cara kerja: BFS membangun lapisan augmenting path terpendek; DFS memperbesar matching pada lapisan itu hingga habis.

Pakai: HopcroftKarp hk(L,R); hk.g[u].push_back(v); int m=hk.run(); hk.a[u] adalah pasangan kanan atau -1. ID kiri dan kanan masing-masing mulai 0.

```
struct HopcroftKarp {
  int L, R, lim;
  vector<vector<int>> g;
  vector<int> a, b, d;
  HopcroftKarp(int L, int R)
      : L(L), R(R), g(L), a(L, -1), b(R, -1), d(L) {}
  bool bfs() {
    queue<int> q;
    lim = INT_MAX;
    fill(d.begin(), d.end(), -1);
    for (int u = 0; u < L; u++)
      if (a[u] < 0) {
        d[u] = 0;
        q.push(u);
      }
    while (!q.empty()) {
      int u = q.front();
      q.pop();
      if (d[u] >= lim) continue;
      for (int v : g[u]) {
        int w = b[v];
        if (w < 0)
          lim = d[u] + 1;
```

Hopcroft-Karp (lanjutan)

```
        else if (d[w] < 0) {
          d[w] = d[u] + 1;
          q.push(w);
        }
      }
    }
    return lim != INT_MAX;
  }
  bool dfs(int u) {
    for (int v : g[u]) {
      int w = b[v];
      if ((w < 0 && d[u] + 1 == lim) ||
          (w >= 0 && d[w] == d[u] + 1 && dfs(w))) {
        a[u] = v;
        b[v] = u;
        return true;
      }
    }
    d[u] = -1;
    return false;
  }
  int run() {
    int ans = 0;
    for (int v : a)
      ans += (v >= 0);
    while (bfs())
      for (int u = 0; u < L; u++)
        if (a[u] < 0 && dfs(u)) ans++;
    return ans;
  }
};
```

Reduksi yang berguna

DAG minimum vertex-disjoint path cover = V - maximum matching pada duplikasi kiri/kanan. Untuk 2021-G: kompres SCC; ambil SCC bertreasure; hubungkan pasangan jika REACHABLE melalui DAG (bukan hanya edge langsung). Teleport minimum = jumlah SCC treasure - matching - 1.

Matching dan cover

Pada bipartite graph, minimum vertex cover size = maximum matching (Konig). Mulai alternating paths dari kiri yang unmatched: cover=(kiri tidak visited)+(kanan visited). Maximum independent set = V-cover.

Dinic maximum flow

Fungsi / syarat: Cap nonnegatif; O(V^2 E) umum. DFS rekursif.

Cara kerja: BFS membuat level graph pada residual network. DFS mengirim blocking flow; edge balik memungkinkan membatalkan aliran sebelumnya.

Pakai: Dinic f(n); f.add(u,v,cap); ll ans=f.flow(s,t); s!=t. Kapasitas tersisa berada di f.g. Panggilan flow berikutnya menghitung aliran TAMBAHAN, bukan total dari awal.

```
struct Dinic {
  struct E {
    int v, rev;
    ll cap;
  };
  vector<vector<E>> g;
  vector<int> d, it;
  Dinic(int n) : g(n), d(n), it(n) {}
  void add(int u, int v, ll c) {
    assert(u != v);
    int a = g[u].size(), b = g[v].size();
    g[u].push_back({v, b, c});
    g[v].push_back({u, a, 0});
  }
  bool bfs(int s, int t) {
    fill(d.begin(), d.end(), -1);
    queue<int> q;
    d[s] = 0;
    q.push(s);
    while (!q.empty()) {
      int u = q.front();
      q.pop();
      for (auto e : g[u])
        if (e.cap && d[e.v] < 0) {
          d[e.v] = d[u] + 1;
          q.push(e.v);
        }
    }
    return d[t] >= 0;
  }
  ll dfs(int u, int t, ll f) {
    if (u == t) return f;
    for (int &i = it[u]; i < (int)g[u].size(); i++) {
      E &e = g[u][i];
      if (e.cap && d[e.v] == d[u] + 1) {
        ll z = dfs(e.v, t, min(f, e.cap));
        if (z) {
          e.cap -= z;
          g[e.v][e.rev].cap += z;
          return z;
        }
      }
    }
    return 0;
```

Dinic maximum flow (lanjutan)

```
  }
  ll flow(int s, int t) {
    assert(s != t);
    ll ans = 0;
    while (bfs(s, t)) {
      fill(it.begin(), it.end(), 0);
      while (ll z = dfs(s, t, INF))
        ans += z;
    }
    return ans;
  }
};
```

Trie string: insert dan terminal id

Fungsi / syarat: Alphabet lowercase a-z; O(total panjang), memory 26 per node.

Cara kerja: Satu edge mewakili satu karakter; kata yang berbagi prefiks berbagi node. Node akhir menyimpan id kata terakhir yang dimasukkan.

Pakai: WordTrie tr; tr.add("abc",7); cari dari u=0, ikuti tr.t[u].to[c-'a']; 0 berarti edge tidak ada. Setelah seluruh kata, tr.t[u].id!=-1 menandakan kata lengkap.

```
struct WordTrie {
  struct N {
    array<int, 26> to{};
    int id = -1;
  };
  vector<N> t = vector<N>(1);
  void add(const string &s, int id) {
    int u = 0;
    for (char c : s) {
      int k = c - 'a';
      if (!t[u].to[k]) {
        t[u].to[k] = t.size();
        t.push_back({});
      }
      u = t[u].to[k];
    }
    t[u].id = id;
  }
};
```

DP dasar, urutan & state kecil

0/1 knapsack & subset sum

Fungsi / syarat: O(nW); iterasi kapasitas MENURUN untuk item sekali.

Cara kerja: dp[c] menyimpan nilai maksimum atau ketercapaian sum. Iterasi menurun memastikan item yang sedang diproses tidak dipakai dua kali.

Pakai: knapsack({2,3},{4,5},3) -> 5. subsetSum({2,3},5)[5] -> true; hasilnya tabel sum 0..S. Bobot/elemen nonnegatif; nilai vector w dan v harus seukuran.

```
ll knapsack(const vector<int> &w,
            const vector<ll> &v,
            int W) {
  vector<ll> dp(W + 1, 0);
  for (int i = 0; i < (int)w.size(); i++)
    for (int j = W; j >= w[i]; j--)
      ckmax(dp[j], dp[j - w[i]] + v[i]);
  return dp[W]; // bobot <= W, subset kosong boleh
}
vector<char> subsetSum(const vector<int> &a, int S) {
  vector<char> dp(S + 1);
  dp[0] = 1;
  for (int x : a)
    for (int s = S; s >= x; s--)
      dp[s] |= dp[s - x];
  return dp;
}
```

LIS dengan rekonstruksi

Fungsi / syarat: Strictly increasing; O(n log n).

Cara kerja: tail[len] adalah ujung terkecil subsequence panjang len+1. Binary search memperbarui tail; parent merekonstruksi indeks.

Pakai: auto ids=lisIndices({3,1,2}); ids -> {1,2}. Panjang LIS=ids.size(); cetak a[id] untuk nilai subsequence. Ini bukan subarray bersebelahan.

```
vector<int> lisIndices(const vector<ll> &a) {
  vector<ll> tail;
  vector<int> at, par(a.size(), -1);
  for (int i = 0; i < (int)a.size(); i++) {
    int k =
        lower_bound(tail.begin(), tail.end(), a[i]) -
        tail.begin();
    if (k) par[i] = at[k - 1];
    if (k == (int)tail.size()) {
      tail.push_back(a[i]);
      at.push_back(i);
    } else {
      tail[k] = a[i];
      at[k] = i;
    }
  }
  vector<int> ans;
  for (int u = at.empty() ? -1 : at.back(); u >= 0;
       u = par[u])
    ans.push_back(u);
  reverse(ans.begin(), ans.end());
  return ans;
}
```

LCS panjang, memory O(m)

Fungsi / syarat: O(nm), string/sequence; traceback perlu tabel/parent.

Cara kerja: Jika karakter sama, tambah 1 dari diagonal sebelumnya; jika berbeda, ambil maksimum membuang karakter kiri atau kanan.

Pakai: lcsLength("abc","ac") -> 2. Hasil hanya panjang subsequence bersama, bukan substring dan bukan string rekonstruksinya; memory O(|b|).

```
int lcsLength(string a, string b) {
  vector<int> d(b.size() + 1);
  for (char x : a) {
    int diag = 0;
    for (int j = 1; j <= (int)b.size(); j++) {
      int old = d[j];
      d[j] = (x == b[j - 1]) ? diag + 1
                             : max(d[j], d[j - 1]);
      diag = old;
    }
  }
  return d.back();
}
```

DP interval & counting

Interval: loop len=1..n, l, r=l+len; transisi split k di (l,r). Coin change jumlah kombinasi: loop coin luar, sum naik di dalam. Jumlah urutan: loop sum luar, coin di dalam. DP max dengan negatif: jangan inisialisasi state unreachable ke 0.

Digit DP: jumlah digit sum divisible K

Fungsi / syarat: Hitung x dalam [0,N], termasuk 0; K>=1. O(digit*K*10).

Cara kerja: State menyimpan sisa jumlah digit mod K dan apakah prefiks masih sama dengan N; tiap digit memperbarui kedua state.

Pakai: digitSumDiv(20,3) -> 7, termasuk 0. Untuk [L,R], hitung digitSumDiv(R,K)-digitSumDiv(L-1,K); yang dibagi K adalah JUMLAH DIGIT.

Digit DP: jumlah digit sum divisible K (lanjutan)

```
ll digitSumDiv(ll N, int K) {
  if (N < 0) return 0;
  string s = to_string(N);
  vector<vector<ll>> d(2, vector<ll>(K));
  d[1][0] = 1;
  for (char c : s) {
    vector<vector<ll>> nd(2, vector<ll>(K));
    for (int t = 0; t < 2; t++)
      for (int r = 0; r < K; r++) {
        int lim = t ? c - '0' : 9;
        for (int x = 0; x <= lim; x++)
          nd[t && x == lim][(r + x) % K] += d[t][r];
      }
    d.swap(nd);
  }
  return d[0][0] + d[1][0];
}
```

DP minimum & maksimum setelah operasi

Fungsi / syarat: 2022-G: kedua ekstrem diperlukan saat pengali negatif.

Cara kerja: Setiap operasi monoton mencapai ekstrem pada lo atau hi. Uji kedua ujung untuk seluruh pilihan operasi pada langkah berikutnya.

Pakai: auto [lo2,hi2]=extremaStep(lo,hi,ops); ops berisi {operator,angka}, satu dipilih pada langkah ini. Mulai lo=hi=nilai awal, lalu ulangi per langkah; ini bukan menjalankan semua ops berurutan.

```
pair<ll, ll> extremaStep(
    ll lo, ll hi, const vector<pair<char, ll>> &ops) {
  ll nlo = INF, nhi = -INF;
  for (auto [op, c] : ops)
    for (ll x : {lo, hi}) {
      ll y = (op == '+') ? x + c : x * c;
      ckmin(nlo, y);
      ckmax(nhi, y);
    }
  return {nlo, nhi};
}
```

Optimasi DP yang sering dipakai

Transisi jumlah dp[j] pada j di range: prefix sum/sliding sum. Transisi min/max dp[j] pada window: monotonic deque. State masa lalu hanya satu layer: rolling array. Pilihan dengan biaya linear mx+b: Li Chao. Divide-and-conquer DP butuh argmin monoton yang dibuktikan.

KHUSUS 2021-B: Wooden Fence

Fungsi / syarat: Panjang 2 boleh vertikal; sisanya dibagi dua baris.

Cara kerja: Pisahkan batang panjang 2 yang dapat dipasang vertikal. Subset-sum batang lain menentukan apakah selisih dua baris dapat ditutup batang 2.

Pakai: woodenFence(a) -> bool: true bila konfigurasi pagar mungkin. a berisi seluruh panjang batang, semuanya positif. Dependensi subsetSum; bukan algoritma partition umum tanpa aturan batang 2.

```
bool woodenFence(const vector<int> &a) {
  int twos = count(a.begin(), a.end(), 2), sum = 0;
  vector<int> rest;
  for (int x : a)
    if (x != 2) {
      rest.push_back(x);
      sum += x;
    }
  if (sum & 1) return false;
  auto dp = subsetSum(rest, sum);
  for (int s = 0; s <= sum; s++)
    if (dp[s]) {
      // selisih dua baris ditutup board horizontal 2
      if (abs(sum - 2 * s) <= 2 * twos) return true;
    }
  return false;
}
```

Bitmask, profile DP, SOS & game

Held-Karp: lintasan mengunjungi subset

Fungsi / syarat: dp[mask][last]; d nonnegatif/INF. O(k^2 2^k).

Cara kerja: State (mask,last) menyimpan biaya minimum mengunjungi tepat subset mask dan berakhir di last; tambah satu tujuan belum dikunjungi.

Pakai: auto dp=subsetPaths(d,start); min(dp.back()) adalah biaya mengunjungi semua titik. d[i][j] jarak antartujuan; start[i] biaya awal menuju i. Subset kosong tidak memiliki endpoint.

```
vector<vector<ll>>
subsetPaths(const vector<vector<ll>> &d,
            const vector<ll> &start) {
  int k = d.size(), N = 1 << k;
  vector<vector<ll>> dp(N, vector<ll>(k, INF));
  for (int i = 0; i < k; i++)
    dp[1 << i][i] = start[i];
  for (int s = 1; s < N; s++)
    for (int u = 0; u < k; u++)
      if (dp[s][u] < INF)
        for (int v = 0; v < k; v++)
          if (!(s >> v & 1))
            if (d[u][v] < INF)
              ckmin(dp[s | 1 << v][v],
                    dp[s][u] + d[u][v]);
  return dp;
}
```

SOS / subset zeta transform

Fungsi / syarat: Setelah transform: a[mask]=sum asal[sub] untuk sub subset mask.

Cara kerja: Untuk setiap bit, tambahkan nilai mask tanpa bit itu ke mask dengan bit tersebut; semua subset dihitung tanpa enumerasi satu per satu.

Pakai: vector<ll> f={1,2,3,4}; sos(f); hasil {1,3,4,10}. sos(f,true) membalik transform. Panjang wajib 2^B; hasil tidak otomatis modulo, jaga overflow.

```
void sos(vector<ll> &a, bool inverse = false) {
  int n = a.size();
  assert(n && !(n & (n - 1)));
  for (int b = 1; b < n; b <<= 1)
    for (int s = 0; s < n; s++)
      if (s & b)
        a[s] += inverse ? -a[s ^ b] : a[s ^ b];
}
```

Meet-in-the-middle: subset sum <= K

Fungsi / syarat: a bisa negatif; O(2^(n/2) log 2^(n/2)).

Cara kerja: Pisahkan array menjadi dua bagian, enumerasi semua jumlah subset masing-masing, lalu binary search pasangan yang totalnya <=K.

Pakai: subsetSums({1,2}) -> {0,1,2,3}; subsetCountLE({1,2},2) -> 3, termasuk subset kosong. Nilai negatif boleh; cocok sekitar n<=40.

```
vector<ll> subsetSums(vector<ll> a) {
  vector<ll> s = {0};
  for (ll x : a) {
    int m = s.size();
    for (int i = 0; i < m; i++)
      s.push_back(s[i] + x);
  }
  return s;
}
ll subsetCountLE(const vector<ll> &a, ll K) {
  int m = a.size() / 2;
  auto x = subsetSums({a.begin(), a.begin() + m});
  auto y = subsetSums({a.begin() + m, a.end()});
  sort(y.begin(), y.end());
  ll ans = 0;
  for (ll v : x)
    ans += upper_bound(y.begin(), y.end(), K - v) -
           y.begin();
  return ans;
}
```

Profile DP: domino pada grid kosong h x w

Fungsi / syarat: h kecil; mask = sel yang sudah terisi dari kolom sebelumnya.

Cara kerja: Mask menandai sel yang terisi domino horizontal dari kolom kiri. DFS membuat transisi kolom; DP menyapu seluruh kolom.

Pakai: dominoTiling(2,3) -> 3 modulo MOD. Grid kosong tanpa hambatan; h harus kecil karena ada 2^h state. Boleh tukar h,w agar h=min(h,w).

```
ll dominoTiling(int h, int w) {
  int S = 1 << h;
  vector<vector<int>> to(S);
  for (int mask = 0; mask < S; mask++) {
    auto dfs =
        [&](auto &&self, int r, int nxt) -> void {
      if (r == h) {
        to[mask].push_back(nxt);
        return;
      }
      if (mask >> r & 1) {
        self(self, r + 1, nxt);
        return;
      }
      self(self, r + 1, nxt | 1 << r); // horizontal
      if (r + 1 < h && !(mask >> (r + 1) & 1))
        self(self, r + 2, nxt); // vertikal
    };
    dfs(dfs, 0, 0);
  }
  vector<ll> dp(S);
  dp[0] = 1;
  for (int c = 0; c < w; c++) {
    vector<ll> nd(S);
    for (int s = 0; s < S; s++)
      for (int t : to[s])
        nd[t] = (nd[t] + dp[s]) % MOD;
    dp.swap(nd);
  }
  return dp[0];
}
```

Game impartial / Grundy

Normal play: tidak bisa bergerak = kalah. Grundy[state]=mex({Grundy[next]}). Komponen independen digabung XOR; xor!=0 berarti menang. Nim: XOR ukuran pile. Misere Nim (yang mengambil terakhir kalah): jika semua pile 1, menang iff jumlah pile genap; selain itu XOR biasa.

KHUSUS Card Game INC 2021-C

Fungsi / syarat: n<=100000, P/Q permutasi 0-based. Tabel khusus soal.

Cara kerja: DSU mengelompokkan siklus dari pasangan permutasi. XOR nilai Grundy khusus tiap ukuran siklus menentukan pemenang.

Pakai: cardGame(p,q) -> true jika pemain pertama menang. Ubah nilai kartu 1..n menjadi 0..n-1; p dan q masing-masing permutasi. Dependensi DSU; tabel hanya valid untuk n<=100000.

```
bool cardGame(const vector<int> &p,
              const vector<int> &q) {
  int n = p.size();
  DSU d(n);
  for (int i = 0; i < n; i++)
    d.join(p[i], q[i]);
  set<int> one = {1,
                  2,
                  5,
                  13,
                  21,
                  31,
                  47,
                  73,
                  99,
                  125,
                  151,
                  177,
                  315,
                  409};
  int nim = 0;
  for (int i = 0; i < n; i++)
    if (d.find(i) == i) nim ^= one.count(d.sz[i]);
  return nim != 0;
}
```

Bitset DP

Jika S batas compile-time: bitset<S+1> dp; dp[0]=1; untuk tiap x lakukan dp |= dp<<x. O(nS/word_size), berguna subset sum. bitset juga mempercepat transitive closure: jika reach[i][k], reach[i] |= reach[k].

Teori bilangan, modular & CRT

Mod power, gcd extended & inverse umum

Fungsi / syarat: mod>0; exgcd a,b>=0. Inverse ada iff gcd(a,mod)=1.

Cara kerja: modpow memakai kuadrat berulang; exgcd melakukan substitusi balik Euclid untuk ax+by=gcd(a,b). Inverse berasal dari koefisien x.

Pakai: modpow(2,10,1000)->24; ll x,y; exgcd(6,9,x,y)->3. invmod(3,10)->7, gagal ->-1. floorDiv(-3,2)->-2; ceilDiv(-3,2)->-1; pembagi harus positif.

```
ll modpow(ll a, ll e, ll mod = MOD) {
  assert(mod > 0 && e >= 0);
  a = (a % mod + mod) % mod;
  ll r = 1 % mod;
  for (; e; e >>= 1, a = (i128)a * a % mod)
    if (e & 1) r = (i128)r * a % mod;
  return r;
}
ll exgcd(ll a, ll b, ll &x, ll &y) {
  if (!b) {
    x = 1;
    y = 0;
    return a;
  }
  ll u, v, g = exgcd(b, a % b, u, v);
  x = v;
  y = u - (a / b) * v;
  return g;
}
ll invmod(ll a, ll m) {
  a = (a % m + m) % m;
  ll x, y;
  if (exgcd(a, m, x, y) != 1) return -1;
  return (x % m + m) % m;
}
ll floorDiv(ll a, ll b) { // b>0
  assert(b > 0);
  return a / b - (a % b < 0);
}
ll ceilDiv(ll a, ll b) { // b>0
  assert(b > 0);
  return a / b + (a % b > 0);
}
```

Generalized CRT dua kongruensi

Fungsi / syarat: x=a mod m; x=b mod n. -1 modulus = tidak ada solusi.

Cara kerja: Tulis x=a+m*t, selesaikan kongruensi untuk t memakai extended gcd; selisih b-a harus habis dibagi gcd(m,n).

Pakai: auto [r,M]=crt(2,3,3,5); hasil {8,15}: semua solusi x=8+15*t. M=-1 berarti mustahil. Dependensi exgcd; modulus positif dan lcm muat ll.

Generalized CRT dua kongruensi (lanjutan)

```
pair<ll, ll> crt(ll a, ll m, ll b, ll n) {
  assert(m > 0 && n > 0);
  a = (a % m + m) % m;
  b = (b % n + n) % n;
  ll x, y, g = exgcd(m, n, x, y);
  if ((b - a) % g) return {0, -1};
  ll q = n / g;
  ll t = (ll)((i128)((b - a) / g) * x % q);
  if (t < 0) t += q;
  i128 mod = (i128)m * q;
  assert(mod <= LLONG_MAX);
  ll r = (ll)(((i128)a + (i128)m * t) % mod);
  return {r, (ll)mod};
}
```

Persamaan Diophantine

a*x+b*y=c punya solusi iff gcd(a,b) membagi c. Dari a*x0+b*y0=g: x=x0*c/g+t*b/g, y=y0*c/g-t*a/g. Cari interval t dari batas x,y memakai floorDiv/ceilDiv, hati-hati koefisien negatif.

Linear sieve: SPF, phi, Mobius

Fungsi / syarat: O(N) waktu/memori; N>=1.

Cara kerja: Linear sieve membentuk setiap komposit melalui faktor prima terkecil, sekaligus memperbarui phi dan Mobius. factor mencoba pembagi sampai akar.

Pakai: Sieve s(100); s.spf[12]=2, s.phi[12]=4, s.mu[12]=0. factor(12) -> {{2,2},{3,1}}. Pecah banyak x<=100 dengan berulang membagi x oleh s.spf[x].

```
struct Sieve {
  vector<int> spf, phi, mu, primes;
  Sieve(int n) : spf(n + 1), phi(n + 1), mu(n + 1) {
    phi[1] = mu[1] = 1;
    for (int i = 2; i <= n; i++) {
      if (!spf[i]) {
        spf[i] = i;
        primes.push_back(i);
        phi[i] = i - 1;
        mu[i] = -1;
      }
      for (int p : primes) {
        if (p > n / i) break;
        int x = i * p;
        spf[x] = p;
        if (i % p == 0) {
          phi[x] = phi[i] * p;
          mu[x] = 0;
          break;
        }
        phi[x] = phi[i] * (p - 1);
```

Linear sieve: SPF, phi, Mobius (lanjutan)

```
        mu[x] = -mu[i];
      }
    }
  }
};
vector<pair<ll, int>> factor(ll n) {
  vector<pair<ll, int>> f;
  for (ll p = 2; p <= n / p; p++)
    if (n % p == 0) {
      int k = 0;
      do {
        n /= p;
        k++;
      } while (n % p == 0);
      f.push_back({p, k});
    }
  if (n > 1) f.push_back({n, 1});
  return f;
}
```

Primitive root untuk modulus prima

Fungsi / syarat: Dipakai 2025-L. Uji pembagi PRIMA dari p-1.

Cara kerja: Order g harus p-1; cukup pastikan g^((p-1)/q)!=1 untuk setiap faktor prima q dari p-1.

Pakai: int g=primitiveRoot(17); hasil 3. Pangkat g^0..g^(p-2) mencakup semua residue nonzero. p wajib prima; dependensi factor dan modpow.

```
int primitiveRoot(int p) {
  if (p == 2) return 1;
  auto fs = factor(p - 1);
  for (int g = 2; g < p; g++) {
    bool ok = true;
    for (auto [q, e] : fs)
      if (modpow(g, (p - 1) / q, p) == 1) {
        ok = false;
        break;
      }
    if (ok) return g;
  }
  return -1; // kontrak: p prima
}
```

Rumus faktor prima

n=product p^e: banyak divisor = product(e+1); jumlah divisor = product(1+p+...+p^e); phi(n)=n*product(1-1/p). sum_{d|n} phi(d)=n. sum_{d|n} mu(d)=[n=1]. Banyak pasangan coprime dapat dihitung sum mu(d)*countMultiples(d)^2.

Harmonic grouping

Untuk semua l dengan n/l sama: q=n/l; r=n/q. Proses interval [l,r], lalu l=r+1. Hanya O(sqrt n) grup. Loop semua kelipatan/divisor hingga N biasanya O(N log N).

Kombinatorika, ekspektasi & matriks

Factorial, nCr, Catalan

Fungsi / syarat: MOD prima; 0<=N<MOD. Build O(N+log MOD), query O(1).

Cara kerja: Precompute factorial dan inverse factorial. C(n,r)=n!/(r!(n-r)!); Catalan memakai selisih dua koefisien binomial.

Pakai: Comb cb(20); cb.C(5,2)->10; cb.catalan(3)->5; cb.f[5]=120. Sediakan kapasitas >=n untuk C dan >=2*n untuk catalan; dependensi modpow.

```
struct Comb {
  vector<ll> f, invf;
  Comb(int n) : f(n + 1, 1), invf(n + 1, 1) {
    assert(n < MOD);
    for (int i = 1; i <= n; i++)
      f[i] = f[i - 1] * i % MOD;
    invf[n] = modpow(f[n], MOD - 2);
    for (int i = n; i > 0; i--)
      invf[i - 1] = invf[i] * i % MOD;
  }
  ll C(int n, int r) const {
    if (r < 0 || r > n || n < 0) return 0;
    assert(n < (int)f.size());
    return f[n] * invf[r] % MOD * invf[n - r] % MOD;
  }
  ll catalan(int n) const {
    return (C(2 * n, n) - C(2 * n, n + 1) + MOD) %
           MOD;
  }
};
```

Desimal ke pecahan eksak

Fungsi / syarat: Tanpa floating point; input nonnegatif, paling banyak 18 digit.

Cara kerja: Baca digit menjadi p/10^d, lalu bagi gcd. Untuk rating, mulai semua nilai di L dan distribusikan sisa agar jumlahnya tepat p.

Pakai: decimalFraction("2.50")-> {5,2}; foodRating("2.5",1,5)-> {4,1}. Vector kosong berarti rata-rata di luar [L,R]. L,R integer; vector rating berukuran denominator setelah reduksi.

```
pair<ll, ll> decimalFraction(string s) {
  ll p = 0, q = 1;
  bool frac = false;
  for (char c : s) {
```

Desimal ke pecahan eksak (lanjutan)

```
    if (c == '.') {
      frac = true;
      continue;
    }
    p = p * 10 + c - '0';
    if (frac) q *= 10;
  }
  ll g = gcd(p, q);
  return {p / g, q / g};
}
vector<ll> foodRating(string x, ll L, ll R) {
  auto [p, q] = decimalFraction(x);
  if ((i128)p < (i128)L * q || (i128)p > (i128)R * q)
    return {};
  vector<ll> a(q, L);
  ll rem = p - L * q;
  for (ll &v : a) {
    ll z = min(rem, R - L);
    v += z;
    rem -= z;
  }
  return a;
}
```

Rumus cepat

1+...+n=n(n+1)/2; sum i^2=n(n+1)(2n+1)/6; sum i^3=[n(n+1)/2]^2. Deret geometri sum_{i=0}^{k-1} a*r^i=a*(r^k-1)/(r-1), tangani r=1. Permutasi nPr=n!/(n-r)!; derangement D0=1,D1=0,Dn=(n-1)(D[n-1]+D[n-2]).

Inclusion-exclusion

|union Ai|=sum nonempty subset S (-1)^(|S|+1)*|intersection Ai, i in S|. Count exact K often = atLeast(K)-atLeast(K+1). EGF memakai coefficient count[k]/k!, perkalian lalu kalikan n! (2022-D).

Matrix exponentiation

Fungsi / syarat: Matriks square; O(d^3 log e), modular.

Cara kerja: Kuadrat berulang pada matriks mempercepat penerapan transisi linear k kali; identitas dipakai untuk pangkat nol.

Pakai: Matrix T={{1,1},{1,0}}; auto R=matPow(T,5); R[0][1]=5 (Fibonacci). matMul(A,B) menghitung A*B; matriks square seukuran, coefficient dinormalisasi modulo MOD.

```
using Matrix = vector<vector<ll>>;
Matrix matMul(const Matrix &a, const Matrix &b) {
  int n = a.size();
  Matrix c(n, vector<ll>(n));
  for (int i = 0; i < n; i++)
    for (int k = 0; k < n; k++)
```

Matrix exponentiation (lanjutan)

```
      if (a[i][k])
        for (int j = 0; j < n; j++)
          c[i][j] =
              (c[i][j] + a[i][k] * b[k][j]) % MOD;
  return c;
}
Matrix matPow(Matrix a, ll e) {
  int n = a.size();
  Matrix r(n, vector<ll>(n));
  for (int i = 0; i < n; i++)
    r[i][i] = 1;
  for (; e; e >>= 1, a = matMul(a, a))
    if (e & 1) r = matMul(r, a);
  return r;
}
```

Ekspektasi & probabilitas modular

E[sum Xi]=sum E[Xi], tidak butuh independence. E[XY]=E[X]E[Y] hanya jika independent. E[X^2]=sum E[Xi^2]+2*sum_{i<j}E[XiXj]. Probabilitas a/b modulo p = a*inverse(b), perlu b%p!=0.

KHUSUS 2022-L: Expected Beauty

Xi = Ai jika Ai=Ai+1, selain itu 0. Untuk uniform independent Ai di interval, E[Xi] = sum t pada irisan dua interval / (len_i*len_i+1). E[Xi^2] memakai sum t^2. Tetangga Xi,Xi+1 berbagi variabel: pakai irisan TIGA interval dan sum t^2. Pasangan terpisah independent; gunakan total sum E[Xi] agar O(n).

Heap & interleaving (2024-L)

Jumlah min-heap dengan label berbeda: H(n)=C(n-1,L)*H(L)*H(R), H(0)=H(1)=1; L,R ukuran subtree complete binary tree. Untuk heap sempurna tinggi h: size=2*size[h-1]+1 dan H[h]=C(size-1,size[h-1])*H[h-1]^2. Jika posisi hasil insert dibatasi, lakukan DP sepanjang ancestor posisi n+1 dan gabung label sibling dengan nCr; tidak cukup hanya H(n).

Pembulatan & pembagian rata

N benda ke k slot seimbang: q=N/k, r=N%k; r slot bernilai q+1, sisanya q. Dot product dua distribusi: untuk minimum, nilai besar dipasangkan dengan kecil (rearrangement). 2024-F: r=N_r%n, s=N_k%n; overlap bagian besar minimum max(0,r+s-n).

Geometri integer & convex hull

Point, cross, segmen & luas

Fungsi / syarat: i128 untuk perkalian koordinat. Contoh |coord|<=1e9.

Cara kerja: Cross menguji orientasi tiga titik. Segmen berpotongan bila ujung berada pada sisi berlawanan, termasuk kasus collinear yang berada dalam bounding box.

Pakai: Point a{0,0},b{2,0},p{1,0}; onSegment(a,b,p)->true. intersects(a,b,c,d) juga true untuk sentuhan ujung/overlap. sign128(cross(a,b,c)) memberi -1,0,+1.

```
struct Point {
  ll x, y;
  int id = 0;
  bool operator<(const Point &p) const {
    return tie(x, y) < tie(p.x, p.y);
  }
};
i128 cross(Point a, Point b, Point c) {
  return ((i128)b.x - a.x) * (c.y - a.y) -
         ((i128)b.y - a.y) * (c.x - a.x);
}
int sign128(i128 x) {
  return (x > 0) - (x < 0);
}
bool onSegment(Point a, Point b, Point p) {
  return cross(a, b, p) == 0 &&
         min(a.x, b.x) <= p.x &&
         p.x <= max(a.x, b.x) &&
         min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}
bool intersects(Point a, Point b, Point c, Point d) {
  int p = sign128(cross(a, b, c)),
      q = sign128(cross(a, b, d));
  int r = sign128(cross(c, d, a)),
      s = sign128(cross(c, d, b));
  if (p * q < 0 && r * s < 0) return true;
  return onSegment(a, b, c) || onSegment(a, b, d) ||
         onSegment(c, d, a) || onSegment(c, d, b);
}
```

Convex hull: semua titik boundary

Fungsi / syarat: Input unik, O(n log n). keep collinear; semua segaris ditangani.

Cara kerja: Sort titik lalu bangun rantai bawah dan atas; buang belokan kanan sehingga boundary menjadi cembung.

Pakai: auto h=hull(points); h berisi titik boundary berurutan, termasuk titik segaris di sisi. Input harus tanpa koordinat duplikat; bila semua segaris hasil berupa urutan titik, bukan polygon berluas positif.

Convex hull: semua titik boundary (lanjutan)

```
vector<Point> hull(vector<Point> p) {
  sort(p.begin(), p.end());
  int n = p.size();
  if (n <= 2) return p;
  bool line = true;
  for (auto q : p)
    if (cross(p[0], p[1], q)) line = false;
  if (line) return p;
  vector<Point> a, b;
  for (auto q : p) {
    while (a.size() > 1 &&
           cross(a[a.size() - 2], a.back(), q) < 0)
      a.pop_back();
    a.push_back(q);
  }
  reverse(p.begin(), p.end());
  for (auto q : p) {
    while (b.size() > 1 &&
           cross(b[b.size() - 2], b.back(), q) < 0)
      b.pop_back();
    b.push_back(q);
  }
  a.pop_back();
  b.pop_back();
  a.insert(a.end(), b.begin(), b.end());
  return a;
}
```

Rumus jarak & lattice

Manhattan = |dx|+|dy|; Chebyshev=max(|dx|,|dy|); Euclidean squared=dx^2+dy^2. Rotasi u=x+y,v=x-y mengubah Manhattan menjadi max(|du|,|dv|). Pick: area=I+B/2-1 untuk polygon sederhana dengan vertex integer; B=sum gcd(|dx|,|dy|).

KHUSUS 2021-H: hull setelah setiap satu titik dihapus

Fungsi / syarat: Unik, id=0..n-1, n>=2. O(n log n), bukan n kali hull.

Cara kerja: Menghapus titik interior tidak mengubah hull. Untuk titik boundary, bangun ulang rantai lokal di antara tetangga hull; beberapa titik batas angular direbuild penuh.

Pakai: Set points[i].id=i; auto ans=hullAfterDelete(points); ans[i] adalah BANYAK titik boundary setelah titik i dihapus. Koordinat unik, n>=2; dependensi Point, cross, hull.

```
vector<int> hullAfterDelete(vector<Point> p) {
  int n = p.size();
  vector<int> ans(n, -1);
  bool line = true;
  for (auto q : p)
    if (cross(p[0], p[1], q)) line = false;
  if (line) return vector<int>(n, n - 1);
```

KHUSUS 2021-H: hull setelah setiap satu titik dihapus (lanjutan)

```
auto it = min_element(p.begin(), p.end());
iter_swap(p.begin(), it);
Point o = p[0];
auto dist = [&](Point q) {
  i128 x = q.x - o.x, y = q.y - o.y;
  return x * x + y * y;
};
sort(p.begin() + 1, p.end(), [&](Point a, Point b) {
  i128 z = cross(o, a, b);
  return z ? z > 0 : dist(a) < dist(b);
});
int k = n - 1;
while (k > 1 && cross(o, p[k - 1], p.back()) == 0)
  k--;
reverse(p.begin() + k, p.end());
vector<Point> h;
for (auto q : p) {
  while (h.size() > 1 &&
         cross(h[h.size() - 2], h.back(), q) < 0)
    h.pop_back();
  h.push_back(q);
}
int H = h.size();
vector<int> pos(n), mark(n);
for (int i = 0; i < n; i++)
  pos[p[i].id] = i;
for (auto q : h)
  mark[q.id] = 1;
for (int i = 0; i < n; i++)
  if (!mark[i]) ans[i] = H;
for (int i = 2; i + 1 < H; i++) {
  vector<Point> st = {h[i - 2], h[i - 1]};
  int l = pos[h[i - 1].id] + 1,
      r = pos[h[i + 1].id];
  for (int j = l; j <= r; j++)
    if (p[j].id != h[i].id) {
      while (st.size() > 1 &&
             cross(st[st.size() - 2],
                   st.back(),
                   p[j]) < 0)
        st.pop_back();
      st.push_back(p[j]);
    }
  ans[h[i].id] = H - 1 + (int)st.size() - 3;
}
// Maksimal 3 titik batas angular: rebuild.
for (int id = 0; id < n; id++)
  if (ans[id] < 0) {
    vector<Point> q;
    for (auto x : p)
      if (x.id != id) q.push_back(x);
```

KHUSUS 2021-H: hull setelah setiap satu titik dihapus (lanjutan)

```
      ans[id] = hull(q).size();
    }
  return ans;
}
```

Sweep kuadran (2021-D, 2023-F)

Untuk kandidat x>=X,y>=Y, minimalkan x+y. Sort x menurun, compress y menurun, insert nilai x+y ke Fenwick prefix-min lalu query y>=Y. Ulangi 4 refleksi/rotasi. Simpan dua minimum dengan id berbeda jika butuh dua nearest neighbor. Update/query dengan x sama harus mengikuti definisi >= atau >.

String: KMP, Z, palindrome & hash

Prefix function & pencarian KMP

Fungsi / syarat: O(|text|+|pattern|); pattern nonempty, overlap didukung.

Cara kerja: Prefix function menyimpan border terpanjang. KMP menggunakan border saat mismatch sehingga karakter text tidak dipindai ulang.

Pakai: kmp("ababa","aba")-> {0,2}, indeks awal 0-based. prefixFunction("abab")-> {0,0,1,2}; pattern harus tidak kosong.

```
vector<int> prefixFunction(const string &s) {
  vector<int> p(s.size());
  for (int i = 1; i < (int)s.size(); i++) {
    int j = p[i - 1];
    while (j && s[i] != s[j])
      j = p[j - 1];
    if (s[i] == s[j]) j++;
    p[i] = j;
  }
  return p;
}
vector<int> kmp(const string &t, const string &p) {
  assert(!p.empty());
  auto pi = prefixFunction(p);
  vector<int> ans;
  int j = 0;
  for (int i = 0; i < (int)t.size(); i++) {
    while (j && t[i] != p[j])
      j = pi[j - 1];
    if (t[i] == p[j]) j++;
    if (j == (int)p.size()) {
      ans.push_back(i - j + 1);
      j = pi[j - 1];
    }
  }
```

Prefix function & pencarian KMP (lanjutan)

```
  return ans;
}
```

Z-function

Fungsi / syarat: z[i] = LCP(s,s.substr(i)); z[0]=n. O(n).

Cara kerja: Simpan interval yang sudah cocok dengan prefix; salin hasil di dalam interval lalu lanjutkan pembandingan hanya bila perlu.

Pakai: zFunction("aaaa")-> {4,3,2,1}. Untuk mencari pattern, hitung Z(pattern+"#"+text); z[i]>=pattern.size() menandakan match, asalkan # tidak muncul pada input.

```
vector<int> zFunction(const string &s) {
  int n = s.size();
  vector<int> z(n);
  if (n) z[0] = n;
  for (int i = 1, l = 0, r = 0; i < n; i++) {
    if (i < r) z[i] = min(r - i, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]])
      z[i]++;
    if (i + z[i] > r) {
      l = i;
      r = i + z[i];
    }
  }
  return z;
}
```

KHUSUS 2023-E: Reverse Severer

Bandingkan prefix target dengan suffix sumber yang dibaca sesuai potongan terbalik. Akumulasi hash dua potongan; saat cocok, greedy reset keduanya dan lanjut. Mencocokkan potongan harus mempertahankan orientasi karakter di dalam potongan. Gunakan dua hash atau verifikasi exact untuk mengurangi risiko collision.

Manacher: radius palindrome ganjil & genap

Fungsi / syarat: O(n); odd[i] termasuk pusat, even[i] berpusat sebelum i.

Cara kerja: Palindrome terluar memberi radius awal melalui pencerminan posisi; ekspansi hanya dilakukan di luar batas yang belum diketahui.

Pakai: auto [odd,even]=manacher(s); isPalindrome(l,r,odd,even) menguji s[l..r-1]. odd[i]=k mencakup [i-k+1,i+k); even[i]=k mencakup [i-k,i+k).

```
pair<vector<int>, vector<int>> manacher(string s) {
  int n = s.size();
  vector<int> odd(n), even(n);
  for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k =
        i > r ? 1 : min(odd[l + r - i], r - i + 1);
```

Manacher: radius palindrome ganjil & genap (lanjutan)

```
    while (i - k >= 0 && i + k < n &&
           s[i - k] == s[i + k])
      k++;
    odd[i] = k--;
    if (i + k > r) {
      l = i - k;
      r = i + k;
    }
  }
  for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = i > r
                ? 0
                : min(even[l + r - i + 1], r - i + 1);
    while (i - k - 1 >= 0 && i + k < n &&
           s[i - k - 1] == s[i + k])
      k++;
    even[i] = k--;
    if (i + k > r) {
      l = i - k - 1;
      r = i + k;
    }
  }
  return {odd, even};
}
bool isPalindrome(int l,
                  int r,
                  const vector<int> &odd,
                  const vector<int> &even) {
  int len = r - l;
  if (len == 0) return true;
  int mid = (l + r) / 2;
  return len & 1 ? odd[mid] >= len / 2 + 1
                 : even[mid] >= len / 2;
}
```

Double rolling hash

Fungsi / syarat: Substring O(1), build O(n). Probabilistik: collision mungkin.

Cara kerja: Prefix hash menyimpan polinomial karakter. Kurangi kontribusi prefix kiri lalu normalkan pangkat; dua modulus menurunkan risiko collision.

Pakai: Hash2 h(s); h.get(l,r) memberi pasangan hash substring [l,r). Dua substring diduga sama bila panjang dan hash sama. Untuk membandingkan dua string, gunakan base yang sama.

```
struct Hash2 {
  const ll M1 = 1000000007, M2 = 1000000009;
  ll B;
  vector<ll> p1, p2, h1, h2;
  Hash2(string s, ll base = 911382323) : B(base) {
    int n = s.size();
```

Double rolling hash (lanjutan)

```
    p1 = p2 = vector<ll>(n + 1, 1);
    h1 = h2 = vector<ll>(n + 1);
    for (int i = 0; i < n; i++) {
      int x = (unsigned char)s[i] + 1;
      p1[i + 1] = p1[i] * B % M1;
      p2[i + 1] = p2[i] * B % M2;
      h1[i + 1] = (h1[i] * B + x) % M1;
      h2[i + 1] = (h2[i] * B + x) % M2;
    }
  }
  pair<ll, ll> get(int l, int r) const {
    return {
        (h1[r] - h1[l] * p1[r - l] % M1 + M1) % M1,
        (h2[r] - h2[l] * p2[r - l] % M2 + M2) % M2};
  }
};
```

Suffix array, LCP & ranking substring

Suffix array: doubling + counting sort

Fungsi / syarat: O(n log n). Byte input >0; sentinel 0 ditambahkan.

Cara kerja: Mulai dari rank karakter, lalu urutkan pasangan rank untuk panjang 2,4,8,... dengan counting sort hingga seluruh suffix terurut.

Pakai: suffixArray("banana")-> {5,3,1,0,4,2}. Hasil adalah posisi awal, bukan rank; buat rank[sa[i]]=i bila perlu. Byte 0 tidak boleh ada dalam input.

```
vector<int> suffixArray(string s) {
  s.push_back(char(0));
  int n = s.size();
  vector<int> p(n), c(n), cnt(max(n, 256)), np(n),
      nc(n);
  for (unsigned char x : s)
    cnt[x]++;
  for (int i = 1; i < 256; i++)
    cnt[i] += cnt[i - 1];
  for (int i = n - 1; i >= 0; i--)
    p[--cnt[(unsigned char)s[i]]] = i;
  int cls = 1;
  c[p[0]] = 0;
  for (int i = 1; i < n; i++) {
    if (s[p[i]] != s[p[i - 1]]) cls++;
    c[p[i]] = cls - 1;
  }
  for (int len = 1; len < n; len *= 2) {
    fill(cnt.begin(), cnt.begin() + cls, 0);
    for (int i = 0; i < n; i++) {
      np[i] = (p[i] - len + n) % n;
      cnt[c[np[i]]]++;
```

Suffix array: doubling + counting sort (lanjutan)

```
    }
    for (int i = 1; i < cls; i++)
      cnt[i] += cnt[i - 1];
    for (int i = n - 1; i >= 0; i--)
      p[--cnt[c[np[i]]]] = np[i];
    int z = 1;
    nc[p[0]] = 0;
    for (int i = 1; i < n; i++) {
      pair<int, int> a = {c[p[i]],
                          c[(p[i] + len) % n]};
      pair<int, int> b = {c[p[i - 1]],
                          c[(p[i - 1] + len) % n]};
      if (a != b) z++;
      nc[p[i]] = z - 1;
    }
    c.swap(nc);
    cls = z;
  }
  p.erase(p.begin());
  return p;
}
```

Kasai LCP

Fungsi / syarat: lcp[i]=LCP(sa[i],sa[i+1]); O(n).

Cara kerja: LCP suffix tetangga diperbarui dengan memakai ulang hasil sebelumnya dikurangi satu, sehingga total perbandingan linear.

Pakai: auto sa=suffixArray(s); auto lcp=kasai(s,sa); lcp[i] adalah kesamaan awal suffix sa[i] dan sa[i+1]. Panjang lcp=max(0,n-1); sa harus berasal dari s yang sama.

```
vector<int> kasai(const string &s,
                  const vector<int> &sa) {
  int n = s.size();
  if (!n) return {};
  vector<int> rank(n), lcp(max(0, n - 1));
  for (int i = 0; i < n; i++)
    rank[sa[i]] = i;
  for (int i = 0, k = 0; i < n; i++) {
    int r = rank[i];
    if (r == n - 1) {
      k = 0;
      continue;
    }
    int j = sa[r + 1];
    while (i + k < n && j + k < n &&
           s[i + k] == s[j + k])
      k++;
    lcp[r] = k;
    if (k) k--;
  }
  return lcp;
```

Kasai LCP (lanjutan)

```
}
```

LCP antara dua posisi & bandingkan substring

Fungsi / syarat: Dependensi: SparseMin. Query O(1).

Cara kerja: LCP dua suffix adalah minimum LCP tetangga di antara rank keduanya. Setelah LCP, karakter pertama berbeda menentukan urutan substring.

Pakai: SuffixLCP q(s); q.common(i,j) memberi panjang prefix sama suffix i,j. q.compare(i,j,len) -> -1/0/1 untuk substring panjang len. Dependensi suffixArray, kasai, SparseMin.

```
struct SuffixLCP {
  string s;
  vector<int> sa, rank;
  unique_ptr<SparseMin> rmq;
  SuffixLCP(string s)
      : s(s), sa(suffixArray(s)), rank(s.size()) {
    for (int i = 0; i < (int)s.size(); i++)
      rank[sa[i]] = i;
    auto v = kasai(s, sa);
    if (!v.empty()) {
      vector<ll> a(v.begin(), v.end());
      rmq = make_unique<SparseMin>(a);
    }
  }
  int common(int i, int j) const {
    if (i == j) return s.size() - i;
    int a = rank[i], b = rank[j];
    if (a > b) swap(a, b);
    return rmq->get(a, b);
  }
  int compare(int i, int j, int len) const {
    int k = min(len, common(i, j));
    if (k == len) return 0;
    return s[i + k] < s[j + k] ? -1 : 1;
  }
};
```

KHUSUS 2022-H: Sorting Machine

Gabungkan baris dengan separator yang tidak muncul pada alfabet. Semua suffix dimulai pada kolom tertentu punya urutan SA yang sama dengan substring sepanjang len, kecuali tie. Cari batas kiri/kanan kelas substring sama memakai binary search LCP>=len. Query ranking stabil = jumlah substring lebih kecil + jumlah substring sama dengan row<=rowTarget.

Menggabungkan SA dengan persistent tree

Pada urutan SA, buat prefix versi persistent yang menambah posisi (kolom*R+row). Selisih dua versi menghitung posisi di interval SA. Query interval indeks posisi membatasi kolom tetap dan row pada [a,b]. Dengan demikian rank substring dalam subset baris bisa dihitung tanpa menyortir tiap query. Build O(RC log(RC)); query batas + hitung O(log(RC)).

Ranking via sparse doubling

Alternatif untuk equality substring statis: simpan kelas setiap panjang 2^k. Substring [l,l+len) diwakili pasangan (rank[k][l],rank[k][l+len-2^k]) dengan k=floor(log2 len). Equality pasangan exact; untuk lexicographic order rank harus berasal dari pengurutan pasangan yang benar.

Kapan perlu suffix array

Banyak query perbandingan substring, longest repeated substring, distinct substring, lexicographic rank. Satu pattern: KMP/Z lebih kecil. Palindrome: Manacher. Jangan memakai std::string::substr di dalam comparator besar karena alokasi dan perbandingan berulang bisa mahal.

NTT & konvolusi siklik

NTT in-place modulo 998244353

Fungsi / syarat: n power of 2, n<=2^23. O(n log n); root primitif 3.

Cara kerja: NTT mengubah coefficient ke domain frekuensi; perkalian polynomial menjadi perkalian per indeks, kemudian inverse NTT mengembalikan coefficient.

Pakai: poly::mul({1,2},{3,4})-> {3,10,8}. Biasanya panggil mul; ntt(a) dan ntt(a,true) dipakai bila mengelola transform sendiri. Dependensi modpow; output modulo MOD.

```
namespace poly {
using V = vector<int>;
void ntt(V &a, bool inverse = false) {
  int n = a.size();
  assert(n && !(n & (n - 1)) && n <= (1 << 23));
  for (int i = 1, j = 0; i < n; i++) {
    int b = n >> 1;
    for (; j & b; b >>= 1)
      j ^= b;
    j ^= b;
    if (i < j) swap(a[i], a[j]);
  }
  for (int len = 2; len <= n; len *= 2) {
    ll root = modpow(3, (MOD - 1) / len);
    if (inverse) root = modpow(root, MOD - 2);
    for (int i = 0; i < n; i += len) {
      ll w = 1;
```

NTT in-place modulo 998244353 (lanjutan)

```
      for (int j = 0; j < len / 2; j++) {
        int u = a[i + j],
            v = w * a[i + j + len / 2] % MOD;
        a[i + j] =
            (u + v >= MOD) ? u + v - MOD : u + v;
        a[i + j + len / 2] =
            (u - v < 0) ? u - v + MOD : u - v;
        w = w * root % MOD;
      }
    }
  }
  if (inverse) {
    ll inv = modpow(n, MOD - 2);
    for (int &x : a)
      x = x * inv % MOD;
  }
}
V mul(V a, V b) {
  if (a.empty() || b.empty()) return {};
  int need = a.size() + b.size() - 1;
  if (min(a.size(), b.size()) < 32) {
    V c(need);
    for (int i = 0; i < (int)a.size(); i++)
      for (int j = 0; j < (int)b.size(); j++)
        c[i + j] =
            (c[i + j] + 1LL * a[i] * b[j]) % MOD;
    return c;
  }
  int n = 1;
  while (n < need)
    n *= 2;
  a.resize(n);
  b.resize(n);
  ntt(a);
  ntt(b);
  for (int i = 0; i < n; i++)
    a[i] = 1LL * a[i] * b[i] % MOD;
  ntt(a, true);
  a.resize(need);
  return a;
}
} // namespace poly
```

Makna coefficient

c[k]=sum_i a[i]*b[k-i]. Distribusi jumlah dua variabel, counting pasangan, polynomial multiplication. Untuk cyclic convolution panjang D, lipat c[k] ke c[k%D]. Untuk correlation, balik salah satu array dahulu.

Perkalian & pangkat cyclic modulo x^D-1

Fungsi / syarat: D>=1; O(D log D log e). Panjang D tidak harus power of 2.

Cara kerja: Konvolusi biasa dilipat menurut indeks mod D. Pangkat dengan binary exponentiation menghitung distribusi jumlah residue berulang.

Pakai: auto r=poly::cyclicPow({1,1,0},2); hasil {1,2,1}. r[k] menghitung pasangan pilihan {0,1} dengan jumlah mod 3=k. poly::cyclic(a,b,D) melakukan satu perkalian; e>=0.

```
namespace poly {
V cyclic(V a, V b, int D) {
  auto c = mul(a, b);
  V r(D);
  for (int i = 0; i < (int)c.size(); i++)
    r[i % D] = (r[i % D] + c[i]) % MOD;
  return r;
}
V cyclicPow(V a, ll e) {
  int D = a.size();
  assert(D > 0);
  V r(D);
  r[0] = 1;
  for (; e; e >>= 1, a = cyclic(a, a, D))
    if (e & 1) r = cyclic(r, a, D);
  return r;
}
} // namespace poly
```

KHUSUS 2025-L: product menjadi penjumlahan

Fungsi / syarat: Prime p<=1e5; S nonzero mod p. Dependensi primitiveRoot.

Cara kerja: Primitive root memetakan perkalian residue nonzero ke penjumlahan eksponen mod p-1; pangkat konvolusi menghitung seluruh urutan.

Pakai: honourable({1,2},2,5,2)->2: urutan (1,2),(2,1). Parameter S=pilihan, K=target produk, p=prima, len=panjang. S dan K di [1,p); dependensi primitiveRoot dan poly::cyclicPow.

```
int honourable(const vector<int> &S,
               int K,
               int p,
               ll len) {
  int D = p - 1, g = primitiveRoot(p), x = 1;
  vector<int> lg(p);
  for (int e = 0; e < D; e++) {
    lg[x] = e;
    x = 1LL * x * g % p;
  }
  poly::V a(D);
  for (int s : S)
    a[lg[s]]++;
  auto r = poly::cyclicPow(a, len);
  return r[lg[K]];
}
```

KHUSUS 2025-L: product menjadi penjumlahan (lanjutan)

KHUSUS 2021-J: SMPA berulang

Fungsi / syarat: M power of 2 <=2^18; K>=1. O(M log M+M log MOD).

Cara kerja: Satu operasi membentuk semua jumlah pasangan mod M. K kali operasi berarti setiap frekuensi NTT dipangkatkan 2^K.

Pakai: smpa({0,1},2,1)->2, jumlah nilai seluruh hasil operasi modulo MOD. a berisi residue [0,M); K adalah jumlah operasi, bukan panjang array. Dependensi poly::ntt, modpow.

```
int smpa(const vector<int> &a, int M, ll K) {
  poly::V f(M);
  for (int x : a)
    f[x]++;
  poly::ntt(f);
  ll e = modpow(2, K, MOD - 1);
  for (int &x : f)
    x = x ? modpow(x, e) : 0;
  poly::ntt(f, true);
  ll ans = 0;
  for (int i = 0; i < M; i++)
    ans = (ans + 1LL * i * f[i]) % MOD;
  return ans;
}
```

Ukuran transform & precision

Linear convolution butuh n>=len(a)+len(b)-1, dibulatkan ke power of two. NTT exact dalam modulus; FFT complex perlu kontrol rounding dan splitting untuk integer besar. Gunakan __int128 saat menggabungkan hasil beberapa NTT via CRT.

Kapan polinomial dipotong

Jika hanya coefficient <=K dibutuhkan, resize(K+1) sesudah setiap perkalian. Untuk cyclic convolution, jangan memotong sebelum melipat karena coefficient derajat tinggi berkontribusi kembali ke derajat rendah.

Formal power series & generating function

Inverse formal power series

Fungsi / syarat: a[0]!=0; return 1/a mod x^n. Dependensi poly::mul.

Cara kerja: Newton menggandakan jumlah coefficient benar: inverse memakai r*(2-a*r), log mengintegralkan a'/a, exp mengoreksi r dengan 1+a-log(r).

Pakai: poly::inverse({1,MOD-1},4)->{1,1,1,1}. derivative(a)/integral(a) memberi turunan/integral konstanta 0. logarithm(a,n) perlu a[0]=1; exponential(a,n) perlu a[0]=0; n adalah jumlah coefficient, bukan derajat.

```
namespace poly {
V inverse(const V &a, int n) {
  assert(n >= 1 && !a.empty() && a[0]);
  V r = {int(modpow(a[0], MOD - 2))};
  while ((int)r.size() < n) {
    int m = min(n, 2 * (int)r.size());
    V f(a.begin(), a.begin() + min(m, (int)a.size()));
    auto q = mul(f, r);
    q.resize(m);
    for (int &x : q)
      x = x ? MOD - x : 0;
    q[0] = (q[0] + 2) % MOD;
    r = mul(r, q);
    r.resize(m);
  }
  return r;
}
V derivative(const V &a) {
  V r(max(0, (int)a.size() - 1));
  for (int i = 1; i < (int)a.size(); i++)
    r[i - 1] = 1LL * i * a[i] % MOD;
  return r;
}
V integral(const V &a) {
  int n = a.size();
  V inv(n + 2), r(n + 1);
  inv[1] = 1;
  for (int i = 2; i <= n; i++)
    inv[i] =
        MOD - 1LL * (MOD / i) * inv[MOD % i] % MOD;
  for (int i = 0; i < n; i++)
    r[i + 1] = 1LL * a[i] * inv[i + 1] % MOD;
  return r;
}
V logarithm(const V &a, int n) {
  assert(n >= 1 && !a.empty() && a[0] == 1);
  auto r = mul(derivative(a), inverse(a, n));
  r.resize(n - 1);
  return integral(r);
}
V exponential(const V &a, int n) {
  assert(n >= 1 && (a.empty() || a[0] == 0));
```

Inverse formal power series (lanjutan)

```
  V r = {1};
  while ((int)r.size() < n) {
    int m = min(n, 2 * (int)r.size());
    auto q = logarithm(r, m);
    for (int i = 0; i < m; i++)
      q[i] = ((i < (int)a.size() ? a[i] : 0) - q[i] +
              MOD) %
             MOD;
    q[0] = (q[0] + 1) % MOD;
    r = mul(r, q);
    r.resize(m);
  }
  return r;
}
} // namespace poly
```

KHUSUS 2023-L: Numbers Combination

Fungsi / syarat: Product (x+...+x^i); ambil coefficient x^K.

Cara kerja: Geser pilihan 1..i menjadi 0..i-1, ubah produk generating function menjadi log agar perkalian menjadi penjumlahan, lalu ambil exp.

Pakai: numbersCombination(3,4)->2: pilih x1=1, x2 di [1,2], x3 di [1,3], jumlah 4. Return banyak pilihan modulo MOD; dependensi poly::exponential.

```
int numbersCombination(int N, int K) {
  if (K < N || (ll)K > 1LL * N * (N + 1) / 2)
    return 0;
  int d = K - N;
  vector<int> inv(d + 2);
  inv[1] = 1;
  for (int i = 2; i <= d; i++)
    inv[i] =
        MOD - 1LL * (MOD / i) * inv[MOD % i] % MOD;
  poly::V f(d + 1);
  for (int j = 1; j <= d; j++)
    f[j] = 1LL * N * inv[j] % MOD;
  for (int i = 1; i <= N && i <= d; i++)
    for (int j = i; j <= d; j += i)
      f[j] = (f[j] - inv[j / i] + MOD) % MOD;
  return poly::exponential(f, d + 1)[d];
}
```

KHUSUS 2022-D: EGF Robot Upgrades

Fungsi / syarat: n robot, caps sesuai urutan level soal. O(m*n log n).

Cara kerja: EGF menyimpan coefficient dibagi factorial agar pilihan part berlabel menjadi konvolusi; potong derajat sesuai batas tiap level.

Pakai: robotUpgrades(n,caps) -> banyak konfigurasi upgrade mod MOD. caps[i] adalah batas banyak part yang di-upgrade setidaknya i+1 kali; jangan sort caps. Masing-masing 0..n; dependensi Comb, poly::mul.

```
int robotUpgrades(int n, vector<int> caps) {
  Comb cb(n);
  poly::V e(n + 1);
  for (int i = 0; i <= n; i++)
    e[i] = cb.invf[i];
  caps.insert(caps.begin(), n);
  poly::V r = {1};
  for (int i = (int)caps.size() - 1; i >= 0; i--) {
    r = poly::mul(r, e);
    r.resize(caps[i] + 1);
  }
  return 1LL * r[n] * cb.f[n] % MOD;
}
```

Formula generating function

Pilihan 0..c: 1+x+...+x^c=(1-x^(c+1))/(1-x). Tak terbatas: 1/(1-x). Banyak objek independen: product polinomial. Urutan panjang L: P(x)^L. Jumlah modulo D: ring modulo x^D-1. Koefisien [x^k](1-x)^(-n)=C(n+k-1,k).

Formal, bukan analisis real

log/exp di sini adalah deret formal modulo prima. Syarat konstanta wajib dipenuhi. inverse polynomial tidak sama dengan membalik tiap coefficient. Panjang n berarti coefficient 0..n-1, jadi coefficient K membutuhkan n=K+1.

Validasi cepat polinomial

Uji convolution terhadap O(nm) pada array kecil. Uji a*inverse(a)=1 mod x^n; logarithm(exponential(f))=f untuk f[0]=0; exponential(logarithm(g))=g untuk g[0]=1. Jangan lupa kasus n=1, konstanta, nol, serta panjang bukan power of two.

INC 2021: interval constraints & teleport DP

Hitung A dari maksimum window B

Fungsi / syarat: 2021-K. B=-1 berarti tidak diketahui. O((n+m)log(n+m)).

Cara kerja: Window diketahui membatasi maksimum tiap posisi. Kelompokkan posisi berdasarkan batas itu; DP mencatat kemunculan terakhir nilai maksimum agar setiap window terpenuhi.

Pakai: countWindowMax({2},2,2)->3: A={1,2},{2,1},{2,2}. b berisi maksimum window panjang m, -1 untuk unknown; A panjang b.size()+m-1 dan nilainya 1..K. Dependensi LazyAffine.

```
int countWindowMax(const vector<int> &b,
                   int m,
                   int K) {
  int n = b.size(), len = n + m - 1;
  vector<int> cap(len);
  multiset<int> active;
  vector<vector<int>> pos(K + 1);
  for (int i = 0; i < len; i++) {
    if (i < n && b[i] > 0) active.insert(b[i]);
    if (i >= m && b[i - m] > 0)
      active.erase(active.find(b[i - m]));
    cap[i] = active.empty() ? K : *active.begin();
    pos[cap[i]].push_back(i);
  }
  vector<vector<pair<int, int>>> req(K + 1);
  for (int i = 0; i < n; i++)
    if (b[i] > 0) {
      auto &p = pos[b[i]];
      int l = lower_bound(p.begin(), p.end(), i) -
              p.begin();
      int r = lower_bound(p.begin(), p.end(), i + m) -
              p.begin();
      if (l == r) return 0;
      req[b[i]].push_back({r, l + 1});
    }
  ll ans = 1;
  for (int v = 1; v <= K; v++)
    if (!pos[v].empty()) {
      int z = pos[v].size();
      vector<int> need(z + 1);
      for (auto [r, l] : req[v])
        need[r] = max(need[r], l);
      LazyAffine st(z + 1);
      st.upd(0, 1, 0, 1);
      int floor = 0;
      for (int j = 1; j <= z; j++) {
        ll total = st.sum(0, j);
        st.upd(
            0, j, v - 1, 0); // tidak memilih nilai v
```

Hitung A dari maksimum window B (lanjutan)

```
        st.upd(j, j + 1, 0, total); // memilih nilai v
        floor = max(floor, need[j]);
        st.upd(
            0, floor, 0, 0); // wajib hit di interval
      }
      ans = ans * st.sum(0, z + 1) % MOD;
    }
  return ans;
}
```

Concealed Domino: greedy dengan kapasitas

2021-L: kelompokkan (a,?) per baris a; reserve count(?,b) slot di setiap kolom b. Kapasitas awal kolom = M-count(?,b). Proses baris dengan demand terbesar; ambil kolom berbeda dengan kapasitas terbesar, lalu decrement sesudah satu baris selesai. Setelah itu isi (?,b) pada baris yang belum dipakai di kolom b; (?,?) mengambil pasangan kosong. Cek demand baris<=M, kapasitas tidak negatif, dan tidak ada pasangan duplikat. Heap + set; O((N+M)log(N+M)).

Tennis Game

2021-M: total per set bisa K..2K-2 atau bilangan genap >=2K. Jika K=2, jawaban YES iff N>=2S dan N genap. Jika K>=3 dan S>=2, semua N>=K*S mungkin. Jika S=1: N>=K dan (N<=2K-2 atau N genap). Gunakan ll untuk K*S.

Quests from the Queen: cooldown teleport

Fungsi / syarat: 2021-E. d jarak antar [istana,quest...]; k<=16.

Cara kerja: Subset DP menghitung jalan tanpa teleport per kelompok quest. Gabungkan kelompok dengan biaya max(cooldown,waktu jalan), plus segmen awal/akhir istana.

Pakai: questsTeleport(d,S) -> waktu minimum menyelesaikan semua quest lalu pulang. d[0]=jarak istana, d[1..k]=quest; isi matriks dari shortest path, S=cooldown. Dependensi subsetPaths, ckmin; teleport boleh menuju lokasi bebas sesuai soal.

```
ll questsTeleport(const vector<vector<ll>> &d, ll S) {
  int k = (int)d.size() - 1, N = 1 << k;
  vector<vector<ll>> w(k, vector<ll>(k));
  vector<ll> home(k), zero(k);
  for (int i = 0; i < k; i++) {
    home[i] = d[0][i + 1];
    for (int j = 0; j < k; j++)
      w[i][j] = d[i + 1][j + 1];
  }
  auto A = subsetPaths(w, home),
       B = subsetPaths(w, zero);
  vector<ll> a(N, INF), b(N, INF), part(N, INF);
  a[0] = b[0] = part[0] = 0;
  for (int s = 1; s < N; s++)
    for (int i = 0; i < k; i++) {
```

Quests from the Queen: cooldown teleport (lanjutan)

```
      ckmin(a[s], A[s][i]);
      ckmin(b[s], B[s][i]);
    }
  auto add = [](ll x, ll y) {
    return (ll)min((i128)INF, (i128)x + y);
  };
  // Segmen tengah: mulai bebas, berakhir teleport.
  for (int s = 1; s < N; s++) {
    int first = s & -s;
    for (int t = s; t; t = (t - 1) & s)
      if (t & first)
        ckmin(part[s],
              add(max(S, b[t]), part[s ^ t]));
  }
  ll ans = INF;
  for (int i = 0; i < k; i++)
    ckmin(ans,
          add(A[N - 1][i], home[i])); // tanpa tele
  for (int s = 0; s < N; s++) {
    ll end =
        a[(N - 1) ^ s]; // segmen terakhir ke istana
    for (int t = s;; t = (t - 1) & s) {
      ll z = add(max(S, a[t]), part[s ^ t]);
      ckmin(ans, add(z, end));
      if (t == 0) break;
    }
  }
  return ans;
}
```

Mengapa segmentasi bekerja

Segmen yang diakhiri teleport butuh max(S,lama jalan), karena cooldown mulai dari teleport sebelumnya. Segmen pertama mulai di istana; segmen tengah mulai bebas; segmen terakhir dapat berjalan pulang tanpa menunggu. Juga periksa solusi tanpa teleport.

Forming a Team

2021-D: jawaban = lebar+tinggi kotak dua orang + jarak Manhattan minimum titik ketiga ke kotak. Pecah ruang menjadi tengah, empat strip, empat kuadran. Template range count/successor dan sweep min cukup sebagai blok query; exclude kedua id yang sudah dipilih.

INC 2025: observasi & query khusus

Polynomially Constructed: derajat minimum

Fungsi / syarat: Hitung turning points dengan aturan parity plateau. O(n).

Cara kerja: Arah selisih nonzero menentukan naik/turun. Rentang nilai sama memaksa turning point; paritas tambahan menjembatani arah sebelum dan sesudahnya.

Pakai: minimumDegree({1,2,1})->2; {5,5,5}->0. y adalah urutan ordinat, bukan koefisien. Fungsi untuk model konstruksi soal dengan absis meningkat yang dapat dipilih; bukan interpolasi titik x yang ditetapkan.

```
int minimumDegree(const vector<ll> &y) {
  vector<pair<int, int>> nz;
  for (int i = 1; i < (int)y.size(); i++)
    if (y[i] != y[i - 1])
      nz.push_back({i - 1, y[i] > y[i - 1] ? 1 : -1});
  if (nz.empty()) return 0;
  int edges = y.size() - 1;
  int turns =
      nz[0].first + edges - 1 - nz.back().first;
  for (int j = 1; j < (int)nz.size(); j++) {
    int z = nz[j].first - nz[j - 1].first - 1;
    int change = (nz[j].second != nz[j - 1].second);
    turns += z + ((z & 1) != change);
  }
  return turns + 1;
}
```

Rectangular Pool: invariant lokal

Fungsi / syarat: Bad = banyak kotak 2x2 dengan tepat 3 puddle. Toggle O(1).

Cara kerja: Komponen puddle berbentuk persegi panjang iff tidak ada 2x2 dengan tepat tiga sel puddle. Toggle hanya memengaruhi empat kotak lokal.

Pakai: Pools p(grid); p.bad==0 menguji kondisi awal. p.toggle(r,c) membalik sel dan mengembalikan apakah SEMUA pool kini rectangular. Karakter . berarti puddle dan # bukan puddle; grid tidak kosong.

```
struct Pools {
  int n, m, bad = 0;
  vector<string> g;
  Pools(vector<string> a)
      : n(a.size()), m(a[0].size()), g(a) {
    for (int i = 0; i < n; i++)
      for (int j = 0; j < m; j++)
        bad += check(i, j);
  }
  int check(int r, int c) {
    if (r < 0 || c < 0 || r + 1 >= n || c + 1 >= m)
```

Rectangular Pool: invariant lokal (lanjutan)

```
      return 0;
    int cnt = 0;
    for (int i = r; i <= r + 1; i++)
      for (int j = c; j <= c + 1; j++)
        cnt += (g[i][j] == '.');
    return cnt == 3;
  }
  bool toggle(int r, int c) {
    for (int i = r - 1; i <= r; i++)
      for (int j = c - 1; j <= c; j++)
        bad -= check(i, j);
    g[r][c] = (g[r][c] == '.') ? '#' : '.';
    for (int i = r - 1; i <= r; i++)
      for (int j = c - 1; j <= c; j++)
        bad += check(i, j);
    return bad == 0;
  }
};
```

Palindrome Partition

Hitung frekuensi huruf; k=max(1,jumlah frekuensi ganjil). Bentuk satu string kiri dari semua pasangan huruf. Tempatkan satu huruf ganjil sebagai pusat palindrome pertama, pasangkan kiri+pusat+reverse(kiri). Huruf ganjil lain masing-masing jadi palindrome panjang 1.

National Science Olympiad

Sort tuple (-(A+C), B+D, nama). Tanda minus hanya untuk key skor; comparator harus strict (<), jangan <=.

Mediation: agregasi pasangan distance

Fungsi / syarat: a[i]=dist(S1,i), b[i]=dist(S2,i). O(n log n).

Cara kerja: Sort menurut jarak ke S1. Untuk pasangan yang tersisa, nilai min jarak ke S2 dibatasi dari bawah oleh jarak ke S1; BIT menghitung count dan sum tiga rentang.

Pakai: Buat p[i]={dist(S1,i),dist(S2,i)} untuk vertex yang dihitung, lalu mediation(p). Return jumlah cost semua pasangan berbeda, satu kali tiap pasangan. Dependensi Fenwick; jarak harus muat ll.

```
ll mediation(vector<pair<ll, ll>> p) {
  sort(p.begin(), p.end());
  vector<ll> ys;
  for (auto [a, b] : p)
    ys.push_back(b);
  sort(ys.begin(), ys.end());
  ys.erase(unique(ys.begin(), ys.end()), ys.end());
  Fenwick cnt(ys.size()), sum(ys.size());
  auto idx = [&](ll y) {
    return lower_bound(ys.begin(), ys.end(), y) -
```

Mediation: agregasi pasangan distance (lanjutan)

```
           ys.begin();
  };
  for (auto [a, b] : p) {
    cnt.add(idx(b), 1);
    sum.add(idx(b), b);
  }
  ll ans = 0, rem = p.size();
  for (auto [a, b] : p) {
    cnt.add(idx(b), -1);
    sum.add(idx(b), -b);
    rem--;
    if (b <= a) {
      ans += a * rem;
      continue;
    }
    int l = upper_bound(ys.begin(), ys.end(), a) -
            ys.begin();
    int r = idx(b);
    ans += a * cnt.pref(l) + sum.sum(l, r) +
           b * (rem - cnt.pref(r));
  }
  return ans;
}
```

Light Control: shortest subrange query

Fungsi / syarat: Prefix basis bertimestamp + RMQ; indeks query [L,R), 0-based.

Cara kerja: Basis prefix menentukan batas kiri terjauh yang masih dapat membentuk target semua bit 1. Binary search memilih endpoint valid, lalu RMQ mengambil panjang terkecil.

Pakai: LightQueries q(a,M); q.get(L,R) -> panjang subrange terpendek di dalam [L,R) yang subset XOR-nya (1<<M)-1; -1 bila mustahil. Ini subset XOR, bukan XOR seluruh subrange. Dependensi PosBasis, SparseMin.

```
struct LightQueries {
  vector<ll> left, len;
  unique_ptr<SparseMin> st;
  LightQueries(const vector<int> &a, int M) {
    PosBasis basis;
    int target = (1 << M) - 1;
    for (int r = 0; r < (int)a.size(); r++) {
      basis.add(a[r], r + 1);
      int l = basis.latestLeft(target);
      left.push_back(l);
      len.push_back(l ? r + 2LL - l : INF);
    }
    st = make_unique<SparseMin>(len);
  }
  ll get(int L, int R) {
    int j =
```

Light Control: shortest subrange query (lanjutan)

```
        lower_bound(left.begin(), left.end(), L + 1) -
        left.begin();
    if (j >= R) return -1;
    return st->get(j, R);
  }
};
```

Construct BFS Graph: konstruksi O(n+m)

Simulasikan queue urutan U. Vertex berikutnya dapat menjadi anak front jika anak pertama atau ID-nya lebih besar dari anak terakhir front; jika tidak, pop front. Saat vertex baru masuk queue, tambah edge wajib ke front. Sisa edge boleh menghubungkannya ke vertex lain yang masih berada di queue. Sisakan anggaran edge untuk n-1 edge tree; jika kapasitas total tidak cukup atau M<n-1, tidak mungkin. Validasi output dengan menjalankan BFS adjacency terurut.

INC 2022-2024: recurrence & konstruksi

Magic Squares: ambang marginal cost

Fungsi / syarat: 2023-K; a[i]>0, P>=1; return biaya dalam i128.

Cara kerja: Biaya menambah elemen ke-i dari k ke k+1 adalah a[i]*(2*k+1). Binary search marginal ke-P, lalu jumlahkan P marginal termurah.

Pakai: magicSquares({1,2},3)->6: alokasi {2,1}, biaya 1*2^2+2*1^2. Return i128; fungsi meminimalkan sum a[i]*x[i]^2 dengan x[i]>=0 dan sum x=P.

```
i128 magicSquares(const vector<ll> &a, ll P) {
  i128 l = 0, r = (i128)(2 * (i128)P - 1) *
                  *min_element(a.begin(), a.end());
  auto count = [&](i128 t) {
    i128 s = 0;
    for (ll x : a)
      s += min((i128)P, (t + x) / (2 * x));
    return s;
  };
  while (l < r) {
    i128 m = (l + r) / 2;
    if (count(m) >= P)
      r = m;
    else
      l = m + 1;
  }
  i128 cost = 0, num = 0;
  for (ll x : a) {
    i128 k = min((i128)P, (l + x) / (2 * x));
    num += k;
    cost += x * k * k;
  }
```

Magic Squares: ambang marginal cost (lanjutan)

```
  return cost - (num - P) * l;
}
```

Count DFS Tree: window recurrence

Fungsi / syarat: 2024-I; N>=1, S tidak kosong. O(max S+|S|).

Cara kerja: p[N]=1; p[i] menjumlahkan N-1 state sebelumnya. Sliding sum memperbarui recurrence; frekuensi S menggabungkan faktor yang sama memakai pangkat.

Pakai: countDFSTree(N,S) -> jumlah konfigurasi soal mod MOD. N adalah kedalaman semua daun (K pada soal); S berisi daftar jumlah node Ai. Return product f_N(Ai), bukan jumlah tree berlabel; dependensi modpow.

```
int countDFSTree(int N, const vector<int> &S) {
  int hi = max(N, *max_element(S.begin(), S.end()));
  vector<ll> p(hi + 2);
  vector<int> f(hi + 2);
  for (int x : S)
    f[x]++;
  p[N] = 1;
  ll win = N == 1 ? 0 : 1;
  ll ans = *min_element(S.begin(), S.end()) >= N;
  for (int i = N + 1; i <= hi; i++) {
    p[i] = win;
    win = (win + p[i] - p[i - N + 1] + MOD) % MOD;
    if (f[i]) ans = ans * modpow(p[i], f[i]) % MOD;
  }
  return ans;
}
```

Observasi cepat 2022

A: enumerasi tripel, cek max-min<=M. B: proses kebutuhan dari level tinggi ke rendah; ambil supply level terdekat yang masih bisa dinaikkan. F: prefix satu periode + aritmetika remainder pada drift total (tangani syarat drift soal). M: dua tinggi posisi ganjil bertetangga bisa tersambung bila selisih -2,0,2; scan run lalu cek ekstensi ujung.

Observasi cepat 2024

C: kelompokkan sum pasangan bersebelahan; greedy ambil pasangan tak overlap untuk setiap sum. E: diff=(digit berikut-digit sekarang) mod 3; run nol panjang t memberi parity t-1; run nonzero memberi parity t-1 jika sum%3!=0, selain itu t. XOR semua parity.

Konstruksi & greedy 2023/2024

2023-M: uji 0/1/2 potongan cokelat; selain itu 3. 2024-K: pisahkan perjalanan yang melintasi S menurut arah, sort satu jarak dan suffix-max jarak lain. 2024-G: arahkan label besar ke kecil; label sama pakai DFS orientation, lalu VALIDASI banyak node reachable dari setiap node. 2024-M: sort 3N angka; hasil minimum median adalah a[N] (0-based).

Primal Collection: constrained max-heap insertion

Fungsi / syarat: 2024-L. N<=1e5, S=1..N+1, K>=0; O(N log N).

Cara kerja: Posisi insert N+1 menentukan jalur ancestor. Hitung heap pada sibling subtree dengan binomial, lalu DP memilih label ancestor agar S naik tepat K kali.

Pakai: primalCollection(N,S,K) -> banyak max-heap awal yang memenuhi syarat mod MOD. Label awal adalah 1..N+1 kecuali S; S disisipkan di posisi N+1. K=jumlah swap ke parent; dependensi Comb, modpow.

```
int primalCollection(int N, int S, int K) {
  int h = 0;
  while ((1LL << (h + 1)) - 1 <= N)
    h++;
  if (K > h) return 0;
  Comb cb(2 * N + 2);
  vector<int> sz(h + 1, 1), path(h + 1), under(h + 1);
  vector<ll> heap(h + 1, 1);
  for (int i = 1; i <= h; i++) {
    sz[i] = 2 * sz[i - 1] + 1;
    heap[i] = cb.C(sz[i] - 1, sz[i - 1]) *
              heap[i - 1] % MOD * heap[i - 1] % MOD;
  }
  for (int i = 0; i <= h; i++)
    path[i] = (N + 1) >> i;
  auto sibling = [&](int i) {
    if (path[i - 1] & 1) return sz[i - 1];
    return i >= 2 ? sz[i - 2] : 0;
  };
  for (int i = 1; i <= h; i++)
    under[i] = under[i - 1] + 1 + sibling(i);
  ll ans = cb.C(S - 1, under[K]);
  for (int i = 1; i <= h; i++) {
    int level = (path[i - 1] & 1) ? i - 1 : i - 2;
    if (level >= 0) ans = ans * heap[level] % MOD;
    if (i <= K)
      ans =
          ans * cb.C(under[i] - 1, sibling(i)) % MOD;
  }
  if (K < h) {
    vector<ll> dp(N + 2);
    dp[N + 1] = 1;
    for (int i = h - 1; i >= K + 1; i--) {
      vector<ll> nd(N + 2);
      ll sum = 0;
```

Primal Collection: constrained max-heap insertion (lanjutan)

```
      for (int j = N; j >= max(S + 1, under[i] + 1);
           j--) {
        ll ways =
            cb.C(j - under[i - 1] - 2, sibling(i));
        int at = max(j + 1, under[i + 1] + 1);
        sum = (sum + dp[at] * ways) % MOD;
        nd[j] = sum;
      }
      dp.swap(nd);
    }
    ans =
        ans * dp[max(S + 1, under[K + 1] + 1)] % MOD;
  }
  return ans;
}
```

Memilih state yang cukup

Banyak soal INC tidak memerlukan algoritma bernama baru. Simpan statistik yang memang dibutuhkan transisi: min/max (2022-G), count+sum (2023-D), batas blok+AND (2024-H), fungsi 16 input (2025-E), basis+timestamp (2025-H).

Bukti konstruksi

Pisahkan syarat perlu (batas jumlah edge, kapasitas, parity, count) dan konstruksi yang mencapai syarat itu. Setelah membuat kandidat, replay definisi soal untuk memvalidasi. Pseudocode BFS/DFS harus diikuti persis, termasuk urutan adjacency dan kapan visited diset.

Tambahan CP & merge interval AND

HLD: path tree menjadi O(log n) interval

Fungsi / syarat: Tree connected, root=0; vertex values. Build O(n).

Cara kerja: Anak bersubtree terbesar menjadi heavy child. Setiap heavy chain berurutan dalam array, sehingga path menjadi beberapa interval segment tree.

Pakai: HLD h(g); isi st.setval(h.pos[u],value[u]). ll ans=0; h.path(u,v,[&](int l,int r){ans+=st.get(l,r);}); ans adalah sum vertex path. Dependensi SegTree+SumOp untuk contoh; g tanpa bobot.

```
struct HLD {
  vector<int> p, dep, sz, heavy, head, pos;
  HLD(const vector<vector<int>> &g) {
    int n = g.size();
    p.assign(n, -1);
    p[0] = 0;
    dep.resize(n);
    sz.assign(n, 1);
```

HLD: path tree menjadi O(log n) interval (lanjutan)

```
    heavy.assign(n, -1);
    head.resize(n);
    pos.resize(n);
    vector<int> ord = {0};
    for (int i = 0; i < (int)ord.size(); i++) {
      int u = ord[i];
      for (int v : g[u])
        if (p[v] < 0) {
          p[v] = u;
          dep[v] = dep[u] + 1;
          ord.push_back(v);
        }
    }
    for (int i = n - 1; i > 0; i--) {
      int v = ord[i], u = p[v];
      sz[u] += sz[v];
      if (heavy[u] < 0 || sz[v] > sz[heavy[u]])
        heavy[u] = v;
    }
    int timer = 0;
    vector<int> st = {0};
    while (!st.empty()) {
      int h = st.back();
      st.pop_back();
      for (int u = h; u != -1; u = heavy[u]) {
        head[u] = h;
        pos[u] = timer++;
        for (int v : g[u])
          if (p[v] == u && v != heavy[u])
            st.push_back(v);
      }
    }
  }
  template <class F> void path(int u, int v, F use) {
    while (head[u] != head[v]) {
      if (dep[head[u]] < dep[head[v]]) swap(u, v);
      use(pos[head[u]], pos[u] + 1);
      u = p[head[u]];
    }
    if (dep[u] > dep[v]) swap(u, v);
    use(pos[u], pos[v] + 1);
  }
};
```

Rollback DSU untuk offline dynamic connectivity

Gunakan union by size TANPA path compression. Setiap join simpan (child,parent,old_size); snapshot=history.size(). Rollback pop sampai snapshot. Masukkan setiap edge aktif pada interval waktu [add,remove) ke segment tree waktu; DFS tree waktu: join saat masuk, jawab query leaf, rollback saat keluar. O((E+Q)log Q log V).

D&C optimization & convexity

dp[k][i]=min_{j<i}(dp[k-1][j]+cost(j,i)). Jika argmin monoton, hitung midpoint lalu rekursi dengan batas kandidat yang menyempit; O(kn log n) bila cost O(1). Jangan menerapkan hanya karena ada loop kuadrat; syarat monoton harus dibuktikan.

Li Chao: minimum garis pada koordinat diskret

Fungsi / syarat: X query diketahui dan unik; O(log |X|) insert/query.

Cara kerja: Pada setiap node simpan garis terbaik di titik tengah. Garis yang kalah hanya bisa menang di salah satu sisi, sehingga insertion turun satu cabang.

Pakai: LiChao lc({1,3,5}); lc.add(2,1); lc.add(-1,8); lc.queryIndex(1)->5 (x=3). X wajib terurut unik; queryIndex memakai INDEKS X, hasil i128. Isi minimal satu garis sebelum query.

```
struct LiChao {
  struct Line {
    ll m = 0, b = 0;
    bool empty = true;
    i128 at(ll x) const {
      return empty ? ((i128)1 << 120)
                   : (i128)m * x + b;
    }
  };
  vector<ll> x;
  vector<Line> t;
  LiChao(vector<ll> X) : x(X), t(4 * X.size()) {
    assert(!x.empty());
    assert(is_sorted(x.begin(), x.end()));
  }
  void add(Line f, int p, int l, int r) {
    int m = (l + r) / 2;
    bool left = f.at(x[l]) < t[p].at(x[l]);
    bool mid = f.at(x[m]) < t[p].at(x[m]);
    if (mid) swap(f, t[p]);
    if (r - l == 1) return;
    if (left != mid)
      add(f, p * 2, l, m);
    else
      add(f, p * 2 + 1, m, r);
  }
  void add(ll m, ll b) {
    add({m, b, false}, 1, 0, x.size());
  }
  i128 query(int i, int p, int l, int r) const {
    i128 ans = t[p].at(x[i]);
    if (r - l == 1) return ans;
    int m = (l + r) / 2;
    return min(ans,
               i < m ? query(i, p * 2, l, m)
                     : query(i, p * 2 + 1, m, r));
  }
```

Li Chao: minimum garis pada koordinat diskret (lanjutan)

```
  i128 queryIndex(int i) const {
    return query(i, 1, 0, x.size());
  }
};
```

KHUSUS 2024-H: greedy merge AND

Fungsi / syarat: Gabungkan pasangan adjacent dengan AND terbesar. O(n log n).

Cara kerja: Max-heap memilih pasangan tetangga dengan AND terbesar. Setelah merge, linked list memperbarui tetangga; entri heap lama dibuang saat tidak valid.

Pakai: alchembit({7,3,1})->4: merge 7&3=3 lalu 3&1=1. Return total reward maksimum menurut aturan soal. Array nonempty, nilai nonnegatif; n=1 menghasilkan 0.

```
ll alchembit(vector<int> a) {
  int n = a.size();
  vector<int> pre(n), nxt(n), live(n, 1);
  priority_queue<tuple<int, int, int>> q;
  for (int i = 0; i < n; i++) {
    pre[i] = i - 1;
    nxt[i] = i + 1;
  }
  auto push = [&](int u, int v) {
    if (u >= 0 && v < n) q.push({a[u] & a[v], u, v});
  };
  for (int i = 0; i + 1 < n; i++)
    push(i, i + 1);
  ll ans = 0;
  int merges = 0;
  while (merges < n - 1) {
    auto [w, u, v] = q.top();
    q.pop();
    if (!live[u] || !live[v] || nxt[u] != v) continue;
    if (w != (a[u] & a[v])) continue;
    ans += w;
    merges++;
    a[u] = w;
    live[v] = 0;
    nxt[u] = nxt[v];
    if (nxt[v] < n) pre[nxt[v]] = u;
    push(pre[u], u);
    push(u, nxt[u]);
  }
  return ans;
}
```